

华中科技大学

视觉认知工程课程设计报告

基于脑启发核心集选择的轻量级 VLA 机械臂动作预测

院 系 人工智能与自动化学院

专业班级 人工智能 2304 班

姓 名 李易阳

学 号 U202315168

指导教师 曹治国 肖阳 陆昊

2026 年 6 月 18 日

Project Page:

<https://jaredmccain-ux.github.io/vision-roc-page/>

摘要

视觉-语言-动作模型（Vision-Language-Action, VLA）是具身智能的核心大脑，能够将环境图像与自然语言指令直接映射为机器人的底层控制动作。然而真实 VLA 训练面临严峻的“数据冗余与算力墙”：人类遥操作采集的轨迹中普遍存在大量时间冗余（连续数十帧画面几乎不变）、分布冗余（少数简单动作占比过高）与次优噪声（操作者的犹豫与失误）。直接在全量数据上训练不仅消耗巨大算力，还会因数据分布失衡而损害模型的泛化能力。

受人脑“预测编码（Predictive Coding）”与“网状激活系统（Reticular Activating System, RAS）”两类认知选择机制的启发，本文构建了一套核心集选择框架：以“首帧/末帧/夹物体帧”三类关键帧为锚点（对应 RAS 的注意力聚焦），再基于“成对距离矩阵”做代表点选择（facility-location）选出核心集。框架的核心是帧间距离的度量方式，本文以两种核心度量为主线对比：（1）动作 + 视觉距离（成对惊奇度），在无监督的输入几何空间中以动作特征与 CLIP 视觉特征的加权差异度量两帧的“惊奇度”；（2）梯度距离（CRAIG），先训练一个轻量模型，用每帧的末层梯度特征度量距离、并按代表簇质量为选中样本配权重做加权训练，使选择对齐回归目标。前者是无监督的输入几何度量，后者使选择与模型真实误差对齐。在这两种核心度量之外，本文还进一步把“动作 + 视觉”拆解开来，单独评估仅动作距离与纯视觉距离两条对照以厘清各分量的贡献，并对其中的视觉项再用 SAM 3 分割对比整图与抠图（白底，仅保留目标臂/夹爪/立方体）两种特征源，考察背景干扰对视觉选择信号的影响。文中给出上述各度量与对照的实验对比以展现其差异。

本文在 ALOHA Sim Transfer Cube 数据集上，用冻结权重的 CLIP 离线提取图文特征，构建“视觉 + 语言 $\rightarrow$ 7 自由度动作”的回归任务，以轻量级多层感知机（MLP）的测试均方误差（MSE）为指标，并对每种方法跑 5 个训练种子报告 $\text{mean} \pm \text{std}$ 以排除随机性。由于右臂 MSE 量级约为左臂的 3~4 倍，两臂“平均”统一采用相对随机的比值以避免被高量级右臂主导。实验表明：两种核心度量中梯度距离（CRAIG）综合最优——仅用 10% 数据，它在动作更剧烈、分布重尾的右臂上 MSE 相对随机降低 6.9%，是唯一真正赢下重尾右臂的度量，两臂平均亦优于随机并逼近全量；输入几何度量（动作 + 视觉）则只能在平稳的左臂占优、在右臂上劣于随机。进一步的分量消融显示仅动作、纯视觉单独使用均

不及二者合用；而对视觉项用 SAM 3 分割抠掉背景后选点更准、使两臂平均比值反超随机（动作 + 视觉 + 抠图取得所有实验的最佳平均），但仍无法让输入几何在重尾右臂上占优。有关本次课设更加详细的中间实验结果可以在本课设的 project page（<https://jaredmccain-ux.github.io/vision-roc-page/>）中查看。

关键词：视觉-语言-动作模型；脑启发核心集选择；数据修剪

目 录

摘要	I
1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 视觉-语言-动作模型与世界-动作模型	2
1.2.2 核心集选择与数据修剪	2
1.2.3 类脑选择机制	3
1.3 主要研究内容	4
1.4 本文组织结构	5
2 问题设置与相关理论	6
2.1 任务定义	6
2.2 数据冗余的形式化	6
2.3 对“冗余”的界定与思考	7
2.4 特征表示	7
3 脑启发核心集选择：从输入几何到梯度感知	8
3.1 模型总览	8
3.2 总体框架	8
3.3 关键帧锚点检测	9
3.4 代表点选择（facility-location）	9
3.5 输入几何距离：成对惊奇度（动作 + 视觉）	12
3.6 梯度距离：CRAIG	12
3.7 核心集选择算法及距离度量设计动机	13
4 实验	15
4.1 实验设置	15
4.2 主要结果	15
4.3 实验结果分析	17
4.4 进一步实验：分量消融与视觉项的分割净化	22
4.5 讨论与局限	24

5 结束语	26
5.1 本文主要工作	26
5.2 主要研究结论	26
5.3 认识与体会	27
参考文献	75

1 绪论

1.1 研究背景与意义

视觉-语言-动作模型（Vision-Language-Action, VLA）是当前具身智能（Embodied AI）的核心大脑^[1,2]，它能够将环境图像与自然语言指令直接映射为机器人的底层控制动作，是连接“感知”与“行动”的关键桥梁。随着遥操作（teleoperation）采集范式的普及，研究者得以低成本地收集大量人类演示轨迹用于训练 VLA 模型^[3,4]。

然而，真实的 VLA 训练面临严峻的“数据冗余与算力墙”问题。人类遥操作采集的数据集通常包含三类典型冗余：其一是**时间冗余**，机械臂在等待、悬停或缓慢移动时，连续几十帧画面几乎没有明显变化，这些帧承载的信息高度重复；其二是**分布冗余**，一些简单、常见的动作（如平移、复位）在数据集中占比过高，而真正决定任务成败的关键动作（如抓取、交接）占比很小，导致训练分布严重不均衡；其三是**次优噪声**，即人类操作者在演示过程中的犹豫、抖动与失误。直接在全量数据上训练，不仅消耗巨大算力、不利于在算力受限的端侧设备上部署，还会因数据分布失衡使模型过度拟合冗余模式，进而损害其泛化能力。

大自然在应对海量多模态数据时为我们提供了高效的解决方案。人脑处理极其庞大的感官数据流，却仅需约 20 瓦的能耗，其核心在于大脑具备卓越的**核心集选择（Coreset Selection）与数据修剪机制**：底层的预测编码（Predictive Coding）^[5]并不被动接受所有帧，而是时刻生成预测，只有当实际画面打破预测（即发生显著状态变化、产生较大预测误差）时神经元才会被强烈激活，这相当于自动过滤了无意义的“发呆帧”；而**网状激活系统（Reticular Activating System, RAS）**^[6]会依据当前任务目标过滤背景噪声，引导注意力聚焦于“高信息效用”的瞬间（如机械爪接触物体、动作轨迹突变的瞬间）。

受此启发，本文旨在借鉴人脑的认知选择机制，为 VLA 数据集设计自动化的核心集选择算法，从冗余数据中智能筛选出最具价值的 10% “核心集”，在保证甚至提升模型性能的前提下大幅降低训练成本，从而支撑轻量级、高能效的端侧 VLA 模型训练。本实验对于探索具身智能在资源受限场景下的高效训练具有参考意义。

1.2 国内外研究现状

1.2.1 视觉-语言-动作模型与世界-动作模型

VLA 模型将大规模视觉-语言预训练的知识迁移到机器人控制中，代表性工作如 RT-2^[1] 与 OpenVLA^[2] 展示了强大的指令跟随与泛化能力；在双臂精细操作方向，ACT^[3] 提出了基于动作分块（action chunking）的 Transformer，并配套发布了低成本的 ALOHA 硬件与仿真数据。随着 Open X-Embodiment^[7] 等跨本体大规模数据集的出现，以及 π_0 ^[8] 等基于流匹配（flow matching）的通用策略的提出，VLA 正从“单点模型”走向“可复用基础设施”：以 LeRobot^[4] 与 StarVLA^[9] 为代表的模块化开源代码库，将视觉-语言主干与动作解码头解耦、可独立替换，并在 LIBERO、ROBOTWIN 等基准上提供统一评测接口，显著降低了 VLA 的开发与对比门槛。

近一年来，与 VLA 紧密相关的研究路线——世界-动作模型（World Action Model, WAM）——逐渐兴起。其核心思想是让模型在输出动作的同时联合预测未来观测（场景演化），即“先想象、后行动”，并通过在大规模视频上预训练、再在动作数据上微调来获得泛化能力^[10]。值得注意的是，WAM 与 VLA 正在架构上收敛——以 StarVLA 为代表的代码仓库已能在同一框架内同时支持 WAM 与 VLA 两种范式。然而无论是 VLA 还是 WAM，上述工作大多聚焦于模型架构与训练范式，而对训练数据本身的冗余性与质量问题关注较少——这正是本文切入的角度。

1.2.2 核心集选择与数据修剪

核心集选择旨在从全量数据中挑选一个小子集，使得在子集上训练能逼近在全量上训练的效果。按选择目标可分为两大流派：一类是覆盖/多样性目标，以最远点采样（Farthest Point Sampling, FPS）^[11] 和 k-center 核心集^[12] 为代表，强调所选点尽量铺开、覆盖数据的边界；另一类是代表性目标，以 k-medoids^[13] 和设施选址（facility location）^[14] 为代表，强调所选点贴近数据的真实分布、落在稠密区中心。后者通常可建模为单调子模函数的最大化，借助贪心算法可获得 $(1 - 1/e)$ 的近似保证^[14]，并可用 CELF 懒惰评估^[15] 大幅加速。除上述两类外，还有一类评分/重要性目标方法，依据训练动态为样本打分并优先保留“难/重要”

样本，如遗忘事件（forgetting events）^[16]、EL2N 与 GraNd^[17]；但在高剪枝率下需引入覆盖约束（如 CCS^[18]）以避免丢弃简单却必要的样本。近年来还出现了面向训练目标的**梯度匹配**类方法，如 CRAIG^[19] 与 GRAD-MATCH^[20]，通过匹配子集与全集的梯度来挑选样本。面向机器人与模仿学习的数据选择则更贴近本文场景，如 DataMIL^[21] 借助 datamodels 以验证损失训练数据质量估计器，在跨本体、多任务等启发式方法易失效的设定下筛选子集。值得注意的是，上述各流派的内核——以“覆盖/代表性”刻画分布、以“重要性/惊奇度”度量样本价值、以“梯度匹配”对齐学习目标，乃至持续学习中借经验**重放**（replay）只复习一小部分样本以抗遗忘——与生物神经系统“只对高价值信息分配资源”的选择机制高度呼应；这一类脑视角正是本文核心集选择算法的设计源头。

1.2.3 类脑选择机制

承上节，机器学习中“如何挑出有价值的数据”这一问题，在神经科学里能找到深刻的对应。下面从**价值度量**、**误差驱动的学习**与**注意门控及选择性巩固**三个层面梳理相关研究；对这些研究的调研最终促成了本文的核心集选择算法。

预测编码与“惊奇”——价值的度量。预测编码理论^[5]认为皮层通过自上而下的预测与自下而上的误差信号进行高效编码：神经元并不被动接受所有输入，只有打破预测、携带较大**预测误差**的“惊奇”（surprise）信号才被进一步处理。Friston 的**自由能原理**^[22]进一步将其升华为统一的脑理论——大脑可视为一台“预测机器”，通过最小化预测误差（自由能的变分上界）同时完成感知、学习与行动。在此框架下，Itti 与 Baldi 给出了“惊奇”的**贝叶斯形式化**^[23]：以观测前后信念分布之间的 KL 散度度量一个事件的惊奇度，并通过眼动实验证实人类注意与注视会被高惊奇事件显著吸引。这为本文以**成对惊奇度**（动作 + 视觉距离）度量帧间价值提供了直接依据——差异大、最“出乎预测”的帧更值得保留。

误差驱动的学习——从输入差异到任务价值。进一步地，神经科学发现中脑多巴胺能神经元编码**奖赏预测误差**（reward prediction error）^[24]：实际结果与预期之差驱动突触可塑性，其机制与机器学习中的时序差分/梯度更新高度同构。这一证据提示，“对学习真正有价值”的样本未必是输入空间上最不同的，而是**最能改变模型、产生较大误差**的样本。这正是本文从纯输入几何的惊奇度，进一

步转向**梯度感知（CRAIG）**的生物学动机——以末层梯度（残差驱动）而非外观差异来界定冗余。

注意门控与选择性巩固——聚焦与压缩。在注意层面，选择性视觉注意通过自上而下的任务目标对相互竞争的输入做偏置选择^[25]，网状激活系统^[6]则在更底层依据唤醒状态与任务相关性对感觉流进行门控；二者共同支撑“依据任务目标聚焦于高效用瞬间”，对应本文以关键帧（首帧/末帧/夹物体帧）为**锚点**的注意力聚焦。在记忆层面，互补学习系统理论^[26]指出大脑借助海马—新皮层的交互，仅**选择性地回放并巩固一小部分经验**即可在避免灾难性遗忘的同时获得泛化——这与“从全量数据中筛选小而高价值的核心集”在目标上高度同构。综上，惊奇/误差驱动的价值度量、任务导向的注意门控与选择性记忆巩固，共同为“在数据流中只保留高价值样本”提供了生物学依据，也构成本文核心集选择框架（锚点 + 惊奇度/梯度距离 + 代表点选择）的设计动机。

1.3 主要研究内容

本文以 ALOHA Sim Transfer Cube 数据集为数据源，围绕 VLA 数据的核心集选择展开如下工作：

- **基准测试：**随机抽取 10% 轨迹样本，利用冻结权重的 CLIP 离线提取图文特征，分别对左右臂构建“视觉特征 + 语言指令 $\rightarrow$ 7 自由度机械臂动作”的回归数据集，训练轻量级 MLP 并报告左右臂平均动作预测的 MSE，作为随机基准。
- **脑启发核心集选择框架设计：**摒弃随机抽样，以三类关键帧（首帧/末帧/夹物体帧）为锚点，在成对距离矩阵上做 facility 代表点选择；核心对比**两种帧间距离度量——动作 + 视觉距离**（即成对惊奇度，无监督输入几何）与**梯度距离 CRAIG**（pilot 末层梯度空间 + 代表簇重加权，对齐回归目标），并详细分析二者的性能差异。
- **算法验证：**利用筛选出的 10% 核心集重新训练 MLP，对每种方法跑 5 个训练种子、与随机基准及全量训练对比，验证“高质量的数据子集能够带来更好的模型性能”；并针对左右臂 MSE 量级悬殊的问题，采用**相对随机比值**报告两臂平均。
- **分量消融与视觉净化：**在两种核心度量之外，进一步把“动作 + 视觉”拆解

为仅动作距离与纯视觉距离两条对照、单独评估各分量的贡献，并对视觉项用 SAM 3 概念分割把整图特征替换为**抠图特征**（白底，仅保留目标臂/夹爪/立方体），在仅改变**选择信号**、不改变 MLP 输入的前提下考察背景干扰的影响。

1.4 本文组织结构

第 2 节给出任务定义、数据冗余的形式化描述与核心集选择问题；第 3 节给出模型总览并介绍核心集选择算法与两种核心帧间距离度量（动作 + 视觉、梯度 CRAIG）；第 4 节给出实验设置、多种子对比结果及其理论分析，以及分量消融与 SAM 3 分割净化视觉选择信号的进一步实验；第 5 节进行实验总结。

2 问题设置与相关理论

2.1 任务定义

记一条演示轨迹 (episode) 为一段按时间顺序排列的帧序列。对其中第 i 帧, 给定该时刻的相机图像 x_i 与一条贯穿全任务的统一自然语言指令 ℓ (如“用右臂抓起立方体并交给左臂”), 目标是预测该时刻单臂的 7 自由度动作向量 $a_i \in \mathbb{R}^7$ ——其中前 6 维为机械臂关节角、第 7 维为夹具开合量。本文将该问题建模为一个特征回归任务: 先用冻结的 (训练中参数不更新) 视觉编码器 $\phi(\cdot)$ 与文本编码器 $\psi(\cdot)$ 分别把图像与指令编码为特征向量 $\phi(x_i) \in \mathbb{R}^{512}$ 与 $\psi(\ell) \in \mathbb{R}^{512}$, 拼接后送入可训练的回归器 f_θ (其参数记为 θ) 预测动作:

$$\hat{a}_i = f_\theta([\phi(x_i) \parallel \psi(\ell)]), \quad (2.1)$$

其中 $\hat{a}_i \in \mathbb{R}^7$ 为预测得到的动作, $[\cdot \parallel \cdot]$ 表示向量拼接 (本文算法的设定中拼接后回归器的输入维度为 $512 + 512 = 1024$)。记全部训练帧的下标集合为 $\mathcal{T}$ 、其元素个数 (训练样本数) 为 $|\mathcal{T}|$, 训练目标为最小化预测动作 $\hat{a}_i$ 与真实动作 a_i 之间的均方误差 (Mean Squared Error, MSE):

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \|f_\theta([\phi(x_i) \parallel \psi(\ell)]) - a_i\|_2^2, \quad (2.2)$$

式中 $\|\cdot\|_2$ 为向量的 ℓ_2 范数, $\mathcal{L}(\theta)$ 即回归器在训练集上的平均预测误差。

2.2 数据冗余的形式化

设全量训练帧集合为 $\mathcal{P} = \{1, \dots, N\}$ 。核心集选择的目标是在预算 $B = \lceil \rho N \rceil$ (本文取比例 $\rho = 10\%$) 下挑选子集 $\mathcal{S} \subset \mathcal{P}$, 使得在 $\mathcal{S}$ 上训练得到的模型 $f_{\theta(\mathcal{S})}$ 在独立测试集 $\mathcal{Q}$ 上的性能与全量数据训练得到的模型相当或比全量的性能还要好。模型训练的目标函数如下:

$$\mathcal{S}^* = \arg \min_{|\mathcal{S}|=B} \frac{1}{|\mathcal{Q}|} \sum_{j \in \mathcal{Q}} \|f_{\theta(\mathcal{S})}([\phi(x_j) \parallel \psi(\ell)]) - a_j\|_2^2. \quad (2.3)$$

由于测试误差是在 $\mathcal{Q}$ 上的平均误差, 理想的 $\mathcal{S}$ 应当在尽量减少冗余的情况下让其经验分布尽量逼近全体的真实分布, 而非一味追求覆盖极端样本。正因如此,

实验早期的 FPS 算法比较弱便情有可原。具体细节可以查看实验结果分析的对应章节。

2.3 对“冗余”的界定与思考

“什么样的帧算冗余”直接决定了核心集该怎么选，因此本文先厘清对冗余的理解。最直观的是**表观冗余**：连续多帧画面/动作几乎一样（时间冗余），或某类简单动作反复出现（分布冗余）。按此定义，只要让所选帧在输入空间里“铺得开、贴合分布”即可——这对应本文的输入几何度量（动作 + 视觉距离）。

但随着实验的进行促使我意识到这一定义并不充分：一帧“看起来不一样”未必“对模型有用”（比如视觉上可能会夹杂着背景与另外一只手臂的噪声——第 4.4 节用 SAM 3 分割抠掉这部分干扰做了直接验证），一帧“看起来普通”也可能恰是模型学不好的地方。结合之前的调研以及对实验的分析，本文把冗余进一步界定为**任务冗余（梯度冗余）**——若一帧对模型参数的更新方向与已选帧高度一致，则它是冗余的，哪怕其外观差异很大。这也直接催生了我使用梯度距离（CRAIG）进行冗余的判断，且该方法在重尾右臂上的结果相比随机更好。可以说，本文从输入几何（动作 + 视觉）到梯度（CRAIG）的两种核心度量，正对应着我在实验过程中对冗余从“表观冗余”到“任务冗余”逐步深化的认识。

2.4 特征表示

本文采用 CLIP^[27]（ViT-B/32）作为冻结的图文编码器，图像与文本分别映射为 512 维特征。CLIP 经过大规模图文对比预训练，其视觉特征在语义层面具有良好的判别性，能够以极低成本离线提取。最终回归器的输入为 $512 + 512 = 1024$ 维的拼接特征。

3 脑启发核心集选择：从输入几何到梯度感知

3.1 模型总览

本文的动作预测模型是一条轻量的“感知 $\rightarrow$ 回归”前向通路，结构如图 3-1 所示：对每一帧，取其相机图像 x_i 与统一语言指令 ℓ ，分别送入冻结的 CLIP 图像编码器与文本编码器，得到两个 512 维特征；二者拼接为 1024 维后输入一个轻量级 MLP（ $1024 \rightarrow 256 \rightarrow 128 \rightarrow 7$ ），直接回归预测该时刻单臂的 7 自由度动作（6 维关节角 + 1 维夹具开合）。CLIP 全程冻结、仅做一次离线特征提取，唯一需要训练的是末端这个小 MLP。本文的核心不在于该模型本身，而在于用哪些帧来训练它——即如何从全量训练帧中选出 10% 的高价值核心集（下文各节），图中输入帧即取自所选核心集。

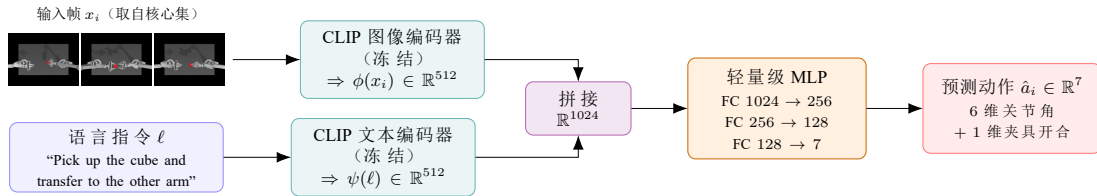


图 3-1 动作预测模型的前向结构。对核心集中的每一帧，相机图像与语言指令分别经冻结的 CLIP 图像/文本编码器得到 512 维特征，拼接为 1024 维后由轻量级 MLP（ $1024 \rightarrow 256 \rightarrow 128 \rightarrow 7$ ）回归 7 自由度动作；CLIP 全程冻结，仅末端 MLP 参与训练。

3.2 总体框架

本文的核心集选择算法由两个共用部件构成：（i）关键帧锚点——为每条 episode 强制保留首帧/末帧/夹物体帧；（ii）在 $N \times N$ 的成对距离矩阵 M 上做 facility-location 代表点选择，直接选出帧数量为 B 帧的核心集。其核心是明确距离矩阵 M 中帧间距离的度量方式，本文以两种核心度量为主线（选点后均等权训练 MLP，仅梯度距离额外按“代表簇质量”为选中样本配权重做加权训练）：

- 动作 + 视觉距离（成对惊奇度）：在无监督的输入几何空间，用机械臂动作特征（含逐维标准化与速度/加速度）与 CLIP 视觉特征余弦差异的加权和度量帧间差异；
- 梯度距离（CRAIG）：先训练一个 pilot 模型，用每帧的末层梯度特征度量距离，并按代表簇质量配权重做加权训练，使选择对齐回归目标。

前者为无监督的输入几何度量、选点后等权训练；后者使选择对齐回归目标、并附带代表簇重加权。下面先给出两种度量共用的锚点与代表点选择（第 3.3–3.4 节），再介绍输入几何度量（动作 + 视觉，第 3.5 节）与梯度距离（CRAIG，第 3.6 节）；二者的实验对比及结果分析放在第 4 节，其中还会把“动作 + 视觉”拆解为仅动作、纯视觉两条对照、并对视觉项做整图/抠图分割净化（第 4.4 节）。

3.3 关键帧锚点检测

对应 RAS 对“高信息效用瞬间”的聚焦，本文为每条 episode 强制保留三类锚点：首帧、末帧与夹物体帧。其中夹物体帧的检测最为关键——它对应机械爪接触并夹到立方体的瞬间，是整个转移任务的“关键”。根据数据集的分析，确定夹具状态由第七维决定（右臂为第十四维，）且值高 = 张开、值低 = 闭合，并且任务流程是“先张开手逼近物体、再合拢夹住”，因此真正的抓取关键帧是一段“张开 → 闭合”的下降沿（而非夹具最张开的峰顶）。具体检测流程见算法 1：在夹具序列上以滞回双阈值确认“张开 → 闭合”的第一段下降沿，取其后续夹得最紧的一帧；对右臂的“先抓后放”，因在第一段闭合即返回，后续的释放段被自动忽略。图 3-2 对全部训练的 45 个 episode 的夹物体帧进行了展示，可见夹爪都准确地夹在了红色立方体上。

3.4 代表点选择（facility-location）

在锚点基础上，算法通过三阶段在距离矩阵 M 上把核心集凑到恰好 B 个帧（算法 2）：阶段一收集每条 episode 的三类锚点；阶段二对三类锚点分别跨 episode 去重——同类锚点在 M 上距离小于全局分位阈值 η 即判为冗余，仅保留索引较小者；阶段三从锚点出发用 facility-location 贪心补齐到 B 。

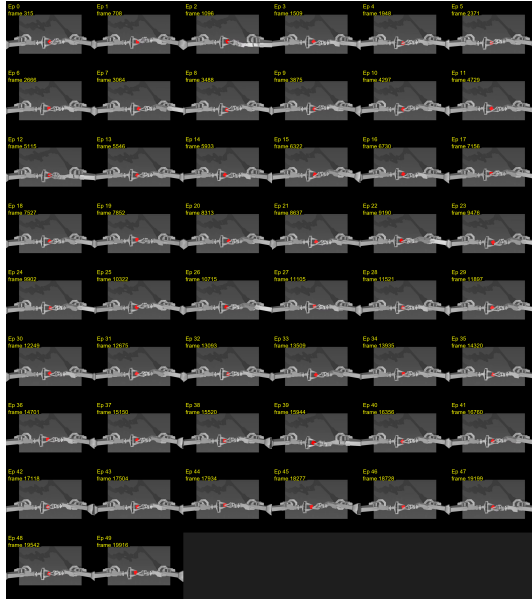
具体地，把距离转为相似度 $s(i, j) = \exp(-(M[i, j]/\sigma)^2)$ (σ 取 M 的中位数)，最大化覆盖函数 $F(\mathcal{S}) = \sum_{i \in \mathcal{P}} \max_{j \in \mathcal{S}} s(i, j)$ （算法 3）：维护每个点的“已覆盖度” $\text{cover}[i]$ ，每轮为候选 c 计算边际收益 $\sum_i \max(0, s(i, c) - \text{cover}[i])$ ，取最大者并更新覆盖度。作为对照，本文也保留了实验早期所采用的最远点采样（FPS, k-center^[11, 12]）作为另一种补齐策略——它每次加入“到已选集合最近距离最大”的点，倾向把预算铺向分布边界。

Algorithm 1 夹物体帧（抓取关键帧）检测

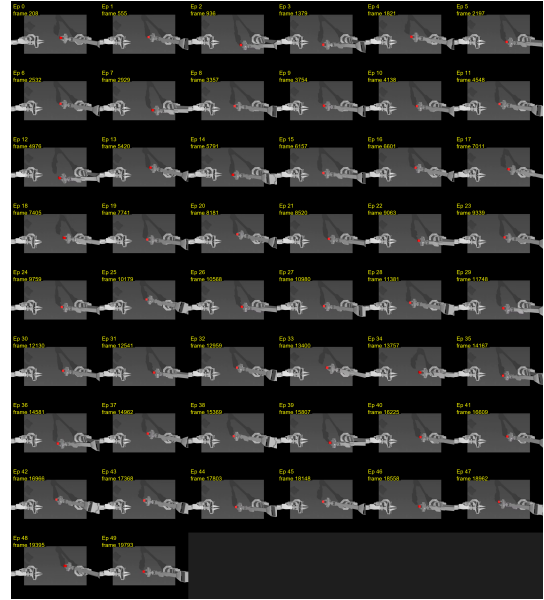
输入：某 episode 的夹具开合时间序列 g （值高 = 张开、值低 = 闭合）；张开/闭合持续帧数 n_o, n_c ；窗口 w

输出：夹物体帧位置 t^* （无明显抓取则返回空）

```
1:  $\tilde{g} \leftarrow$  对  $g$  做滑动平均去噪
2:  $lo \leftarrow Q_{5\%}(\tilde{g})$ ,  $hi \leftarrow Q_{95\%}(\tilde{g})$ ,  $span \leftarrow hi - lo$  // 鲁棒上下界
3: 张开阈值  $\tau_o \leftarrow lo + 0.6span$ ; 闭合阈值  $\tau_c \leftarrow lo + 0.4span$  // 滞回双阈值
4:  $opened \leftarrow \text{false}$ ;  $t_0 \leftarrow \text{空}$ 
5: for  $t = 1, \dots, |g|$  do
6:   if  $\tilde{g}_t \geq \tau_o$  已持续至少  $n_o$  帧 then
7:      $opened \leftarrow \text{true}$  // 确认“张开”
8:   end if
9:   if  $opened$  且  $\tilde{g}_t \leq \tau_c$  且其后保持闭合  $n_c$  帧 then
10:     $t_0 \leftarrow t$ ; 跳出循环 // “张开  $\rightarrow$  闭合”下降沿
11:   end if
12: end for
13: if  $t_0$  为空 then // 夹具全程未见明显抓取
14:   return 空
15: end if
16:  $t^* \leftarrow \arg \min_{t \in [t_0, t_0+w]} \tilde{g}_t$  // 下降沿后夹得最紧的一帧
17: return  $t^*$ 
```



(a) 左臂



(b) 右臂

图 3-2 夹物体帧（抓取关键帧）检测可视化（左右臂各全 50 条 episode）。每格为算法定位到的抓取帧画面，标注其 episode 与全局帧号；可见左右臂的夹爪均准确夹持在红色立方体上。

Algorithm 2 核心集选择主流程（锚点 + 代表点选择）

输入: 成对距离矩阵 M （输入几何或梯度空间），动作 $\{a_i\}$ ，episode 与帧标签，预算 B ，去重阈值 η ，策略 $method$

输出: 核心集索引 $\mathcal{S}$ ， $|\mathcal{S}| = B$

- 1: 对每条 episode 取首帧、末帧、夹物体帧，得三类锚点集合 $\mathcal{A}$ // 阶段一：RAS 注意力聚焦
 - 2: 对三类锚点分别跨 episode 去重：距离 $< \eta$ 则删较大索引者 // 阶段二：去除分布冗余
 - 3: $\mathcal{S} \leftarrow$ 去重后的锚点集合
 - 4: **if** $method = \text{"facility"}$ **then**
 - 5: $\mathcal{S} \leftarrow \text{FACILITYSELECT}(M, \mathcal{S}, \mathcal{P} \setminus \mathcal{S}, B, \sigma)$ // 阶段三：密度感知代表点
 - 6: **else**
 - 7: $\mathcal{S} \leftarrow$ 以 $\mathcal{S}$ 为种子的最远点采样补齐到 B // 阶段三：最远点覆盖
 - 8: **end if**
 - 9: **return** $\mathcal{S}$
-

Algorithm 3 密度感知代表点选择 FACILITYSELECT（facility-location 贪心）

输入: 矩阵 M ，已选 $\mathcal{S}_0$ ，候选 $\mathcal{C}$ ，目标数 B ，带宽 σ

输出: 选中集合 $\mathcal{S}$ ， $|\mathcal{S}| = B$

- 1: 相似度 $s(i, j) \leftarrow \exp(-(M[i, j]/\sigma)^2)$; $\mathcal{S} \leftarrow \mathcal{S}_0$
 - 2: $\text{cover}[i] \leftarrow \max_{j \in \mathcal{S}} s(i, j)$, $\forall i \in \mathcal{P}$ // 已选集合对每个点的当前覆盖
 - 3: **while** $|\mathcal{S}| < B$ **do**
 - 4: 对每个 $c \in \mathcal{C}$: $\text{gain}[c] \leftarrow \sum_{i \in \mathcal{P}} \max(0, s(i, c) - \text{cover}[i])$
 - 5: $c^* \leftarrow \arg \max_c \text{gain}[c]$; $\mathcal{S} \leftarrow \mathcal{S} \cup \{c^*\}$
 - 6: $\text{cover}[i] \leftarrow \max(\text{cover}[i], s(i, c^*))$, $\forall i$ // 更新覆盖度
 - 7: **end while**
 - 8: **return** $\mathcal{S}$
-

3.5 输入几何距离：成对惊奇度（动作 + 视觉）

两种输入几何度量在输入空间构建 M ，对应预测编码的“惊奇度”—— $M[i, j]$ 越大表示两帧越不一样。统一写作

$$M[i, j] = w_a \tilde{d}_{ij}^a + w_v \tilde{d}_{ij}^v, \quad d_{ij}^a = \|\bar{a}_i - \bar{a}_j\|_2, \quad d_{ij}^v = 1 - \cos(\phi(x_i), \phi(x_j)), \quad (3.1)$$

其中 $\bar{a}$ 为对动作逐维标准化并拼接其速度/加速度后的表征（消量纲并引入动力学信息）， d^v 为 CLIP 视觉余弦距离， $\widetilde{(\cdot)}$ 为全局 min-max 归一化（避免数值的不对齐而引入噪声）。本文以动作 + 视觉距离（取 $w_a=0.7, w_v=0.3$ ）作为输入几何的核心度量，称成对惊奇度；它无监督、选择只由输入几何决定。作为分量对照，取 $w_v=0$ 退化为仅动作距离、取 $w_a=0$ 退化为纯视觉距离——这两条对照将在第 4.4 节用于消融，以厘清动作项与视觉项各自的贡献。

3.6 梯度距离：CRAIG

动作 + 视觉距离只看“输入长得有多不同”，与“模型在哪儿真正预测不准”并不对齐。梯度距离即 CRAIG^[19, 20] 改在梯度空间构建 M ，使选择直接对齐回归目标，流程如下。

(1) pilot 模型与梯度特征。先在全量训练池上训练一个与下游同构的 pilot MLP。对每帧前向一次，取预测残差 $e_i = f(\cdot) - a_i$ 与末层输入（隐层激活） h_i ，构造末层梯度特征

$$g_i = [\text{vec}(e_i h_i^\top) \parallel e_i] \in \mathbb{R}^d, \quad (3.2)$$

它正是该帧损失对末层参数的梯度，刻画“这一帧想把模型往哪个方向更新”；以残差驱动这一点恰契合预测编码“误差驱动”的本质。

(2) 梯度距离。将 g_i 单位化后，以方向一致度定义 $M[i, j] = 1 - \widetilde{\cos(g_i, g_j)}$ ：梯度方向一致的两帧要求“同一种修正”、彼此冗余；方向相反或无关的两帧携带不同学习信号。在该 M 上跑第 3.4 节的锚点 + facility 选择得到核心集 $\mathcal{S}$ 。

(3) 代表簇重加权。把全池每帧划给在梯度空间最近的选中点，选中点 j 的权重 w_j 为其名下帧数（代表簇质量），归一到均值 1；以加权 MSE $\sum_{j \in \mathcal{S}} w_j \|f(X_j) - a_j\|^2$ 训练最终模型（算法 4）。

Algorithm 4 梯度感知核心集 CRAIG

输入: 训练池特征 $\{X_i\}$ 与动作 $\{a_i\}$, episode/帧标签, 预算 B

输出: 核心集 $\mathcal{S}$ 及样本权重 $\{w_j\}_{j \in \mathcal{S}}$

- 1: 在全量池上训练 pilot MLP
 - 2: 对每帧取残差 e_i 与隐层 h_i , 构造梯度特征 $\widetilde{g_i} = [\text{vec}(e_i h_i^T) \parallel e_i]$
 - 3: 单位化 $\widetilde{g_i}$, 构建梯度距离 $M[i, j] = 1 - \cos(\widetilde{g_i}, \widetilde{g_j})$
 - 4: 在 M 上以锚点为种子做 facility 选择得 $\mathcal{S}$ (算法 2-3)
 - 5: 把全池每帧划给最近的 $j \in \mathcal{S}$, $w_j \leftarrow$ 其名下帧数, 归一到均值 1
 - 6: 以加权 MSE $\sum_j w_j \|f(X_j) - a_j\|^2$ 训练最终模型
 - 7: **return** $\mathcal{S}, \{w_j\}$
-

3.7 核心集选择算法及距离度量设计动机

本节集中说明前述核心集选择算法及距离度量的设计出发点, 以厘清“为什么这样选”。

从表观冗余到任务冗余: 两种核心度量的动机。本文的两种核心度量并非并列的工程选项, 而对应着对“何为冗余”逐步深化的认识, 其底层动机均源自类脑选择机制。**动作 + 视觉距离 (成对惊奇度)** 立足于**表观冗余**: 若两帧的动作 (及其速度/加速度) 与视觉场景都几乎一致, 则二者高度重复、只需保留其一——这对应预测编码中“画面未打破预测即无需强烈响应”的直觉, 其中动作通道刻画姿态变化、视觉通道补足“动作相近但场景不同” (如同一姿态面对不同物体位形) 的信息, 二者加权即“惊奇度”。然而它停留在**输入空间**, 隐含假设“输入差异大 $\Rightarrow$ 对模型更有用”, 而这一假设在重尾、含噪的子任务上并不成立。**梯度距离 (CRAIG)** 的出发点正是修正这一错位: 把冗余从“看起来像不像”重新界定为**任务冗余**——若一帧对模型参数的更新方向与已选帧高度一致, 则它是冗余的, 哪怕外观差异很大。以末层梯度 (残差驱动) 度量帧间距离, 使选择信号直接对齐回归目标, 这也与预测编码“以预测误差驱动学习”的本质最为贴合。(第 4.4 节还将把“动作 + 视觉”拆为仅动作、纯视觉两条对照, 验证两个输入通道各自的作用。)

FPS 与 facility: 覆盖边界还是贴合分布。锚点确定后的补齐策略, 本文在实验进行过程中采取了包括最远点采样 (FPS) 与代表点选择 (facility-location) 两种策略, 二者出发点截然不同。**FPS** 近似 **k-center** 目标 (最小化最大覆盖半径), 其动机是“让选中点尽量铺开、不留覆盖盲区”, 因而天然偏向分布边界与离群点; 这在追求多样性覆盖时有利, 但在含噪长尾数据上会把宝贵预算耗费在噪

声离群帧上，因此后续实验就转向了 facility-location 策略。**facility** 近似 k-median 目标（最小化各点到最近代表点的距离之和， $\min_S \sum_i \min_{j \in S} d(i, j)$ ），其动机是“让代表点落在稠密区中心、对经验分布做最优量化”，因而偏向高密度的代表性样本。回到式 (2.3)，测试风险是**平均型**指标，奖励的是子集对整体分布的**保真度**而非对极端样本的覆盖；从表 4-1 实验结果来看，FPS 在右臂上劣势明显，两臂平均亦劣于随机基准。

4 实验

4.1 实验设置

数据集：采用 ALOHA Sim Transfer Cube（人类演示）数据集^[3,4]，共 50 条成功演示轨迹、共 20000 帧，任务为“右臂抓起立方体并交给左臂”。每帧含多视角图像、末端执行器姿态与 14 自由度联合动作（单臂 7 自由度）。按 episode 以固定随机种子划分为 45 条训练 / 5 条测试，左右臂共用同一划分，核心集预算为训练帧的 10%。

特征与模型：用冻结的 CLIP（ViT-B/32）离线提取 512 维图像特征与 512 维文本特征，拼接为 1024 维输入。回归器为轻量级 MLP（隐层 256-128，ReLU，Dropout 0.1），Adam 优化（学习率 10^{-3} 、权重衰减 10^{-4} ），批大小 64，训练 200 轮。评价指标为测试集上原始动作空间的均方误差（MSE），越低越好。为排除训练随机性，每种 10% 方法（随机基准与各核心集方法）均以 5 个不同训练种子重复，报告 $\text{mean} \pm \text{std}$ ；左右臂各自评估并取两臂平均。

对比方法：除全量（100%）与随机（10%）两个基准外，核心对比是两种核心距离度量下的核心集（均用锚点 + facility 选择、10% 预算）：动作 + 视觉距离（成对惊奇度）与梯度距离（CRAIG，并配代表簇重加权）；二者的分量对照（仅动作、纯视觉）与视觉项的整图/抠图分割净化作为进一步实验见第 4.4 节。除全量外，每种方法均跑 5 个训练种子，报告 $\text{mean} \pm \text{std}$ 。

4.2 主要结果

表 4-1 与图 4-1 汇总各方法的结果。需要说明的是，最远点采样（FPS）为本实验早期所采用的选择策略；初步实验表明，无论建立在何种输入几何距离上，FPS 在两臂平均（比值口径）上均未能优于随机基准，故本文仅以单种子点估计保留其结果作为对照，未对该策略再做多种子重复实验。后续的 facility 代表点选择与各距离度量则均按 5 个训练种子重复、报告 $\text{mean} \pm \text{std}$ 。

表 4-1 各方法在 ALOHA 转移立方体测试集上的动作预测结果（左右臂为 $\text{MSE} \times 10^{-3}$ ，原始动作空间监督；平均列为两臂相对随机的 MSE 比值之均值——每臂先除以该臂随机基线、再对两臂取平均，以消除左右臂量级差，随机 = 1.000、 < 1 表示优于随机，越低越好）。**facility** 与随机为 5 训练种子 $\text{mean} \pm \text{std}$ ，全量与 FPS 为单种子点估计。加粗为 10% 预算内各列最优。分量对照（仅动作、纯视觉）与整图/抠图分割净化见表 4-2。

方法（除全量外均 10% 预算）	左臂 MSE	右臂 MSE	平均 (比值)
全量（100%）	3.761	11.167	0.931
随机（10%）	$3.901_{\pm 0.42}$	$12.429_{\pm 0.78}$	1.000
最远点采样 FPS（对照）			
仅动作距离	4.223	13.679	1.092
动作 + 视觉距离	4.348	17.012	1.242
<i>facility</i> 代表点选择（两种核心度量）			
动作 + 视觉距离（惊奇度）	$3.387_{\pm 0.23}$	$13.657_{\pm 1.58}$	0.984
梯度距离（CRAIG）	$3.972_{\pm 0.48}$	$11.569_{\pm 0.60}$	0.975

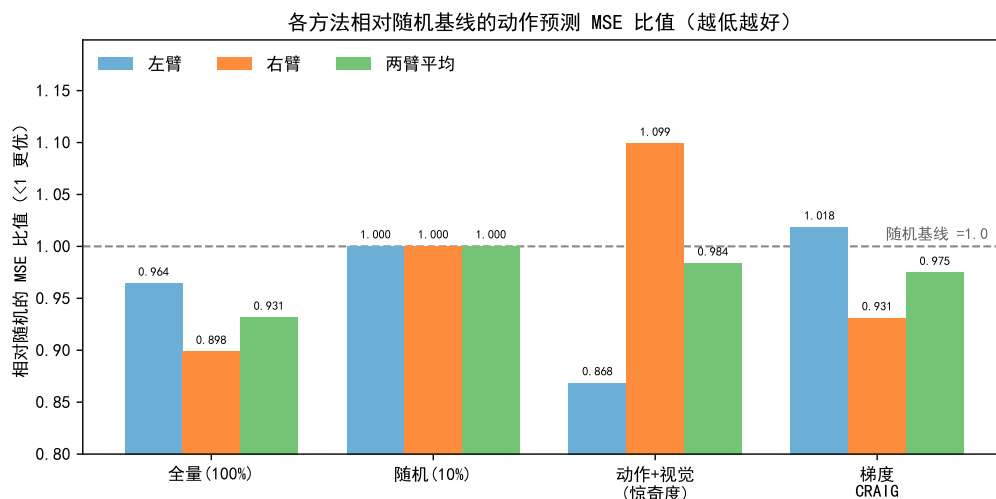


图 4-1 各方法相对随机基线的两臂 MSE 比值（每臂除以该臂随机基线，随机 = 1.0、虚线为基线， < 1 更优）。两种核心度量动作 + 视觉（平均 0.984）与梯度 CRAIG（平均 0.975）的两臂平均比值均跨到随机之下并逼近全量（0.931）；但右臂一栏显示只有 CRAIG（及全量）真正赢下重尾右臂，动作 + 视觉的右臂比值仍 > 1 。

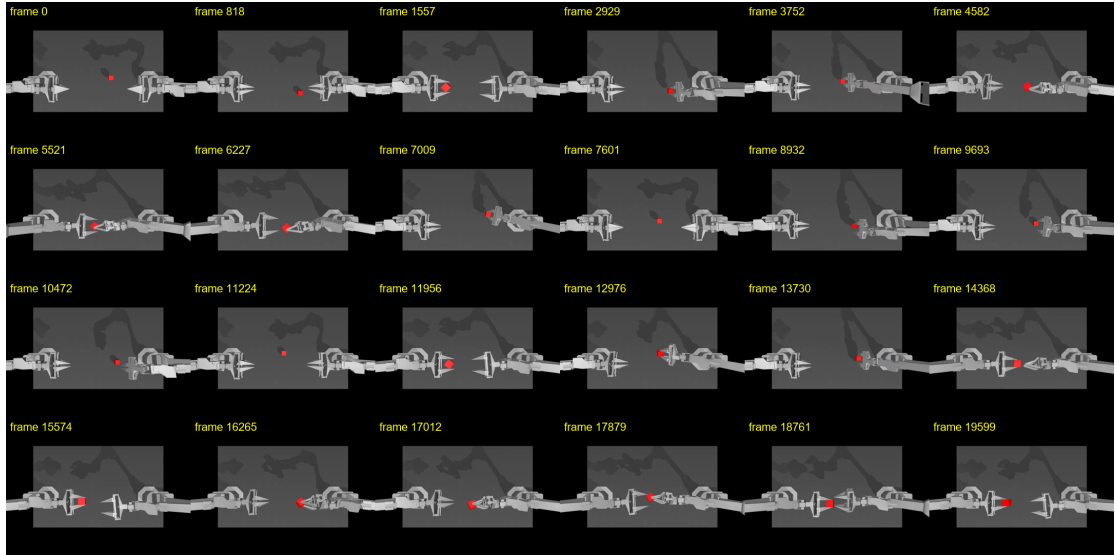


图 4-2 最优方法（梯度距离 **CRAIG**）在右臂上选出的 10% 核心集示例帧（从 1800 帧中均匀抽取 24 帧，标注全局帧号）。所选帧覆盖了从接近、抓取到交接的多种关键姿态。

4.3 实验结果分析

本节结合实验结果分析动作 + 视觉与梯度 **CRAIG** 两种核心度量为何在两臂上表现迥异（并以仅动作距离作对照）：图 4-3 给出左右臂的动作动力学分布；图 4-4 把全部 2×10^4 帧用 CLIP 特征做 2D PCA 投影（灰点），并分别标出四种方法在右臂上最终选中的 10% 核心集（彩点）。

右臂为何更难：如图 4-3，左臂动作高度集中在近零速度（多为接取前的等待/悬停），分布尖锐而轻尾；右臂则在中高速度区（约 $0.02 \sim 0.10$ ）保留了明显更重的“肩部”——它承担主动抓取与交接，动作更剧烈、且夹杂人类遥操作的次优噪声，分布**重尾**。重尾意味着“关键而稀有”的帧更多，对核心集选择更不友好。

为何输入几何度量（动作 + 视觉）只能赢左臂：选择的优劣取决于其偏置是否与“模型需要数据的地方”对齐。采用 $\text{facility}(\text{k-median 目标 } \min_S \sum_i \min_{j \in S} d(i, j))$ 选择，等价于对经验分布做**最优量化**、偏好稠密区代表点（相对地，最远点采样 FPS 近似 **k-center**、被离群点主导，对噪声/长尾敏感——见表 4-1，FPS 在动作 + 视觉距离下右臂 MSE 高达 17.0、明显差于 **facility**）。结合式 (2.3)，测试风险是平均型指标，奖励“分布保真度”。当距离建立在**输入几何**上时，在动作平稳的左臂上几何偏置与数据需求一致、故优于随机；但在重尾含噪的右臂上，输入几何与“模型真实误差”错位——如图 4-4(b)(c)，仅动作与动作 + 视觉选中的帧在

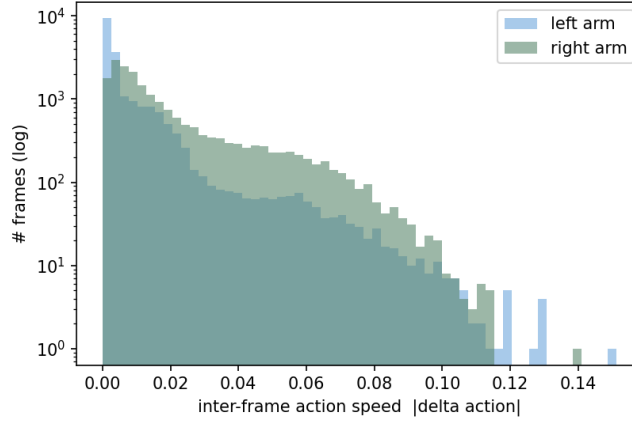


图 4-3 左右臂帧间动作速度 $|\Delta a|$ 的分布（纵轴对数）。左臂尖锐轻尾（多为低速等待），右臂在中高速区保留更重的尾部，动作更剧烈、分布重尾。

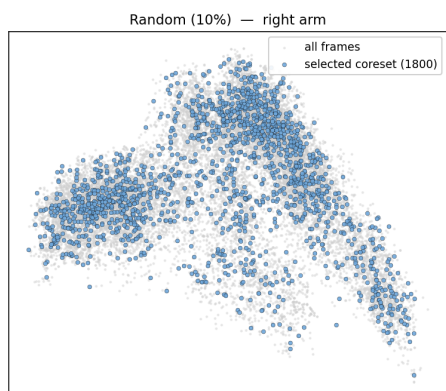
特征流形上分布不均、对部分稠密区覆盖不足而在另一些区域扎堆，与全量分布偏离，故反而劣于无偏的随机抽样（图 4-4(a)）。

为何梯度感知（CRAIG）能赢右臂：CRAIG 用带权子集匹配全量梯度：当 $\sum_{j \in S} w_j \nabla \ell_j \approx \sum_{i \in \mathcal{P}} \nabla \ell_i$ 时，在子集上的一步优化近似全量的一步优化，从而“子集训练 $\approx$ 全量训练”。该信号对每只臂各自的难度自适应：梯度冗余的易帧被折叠进代表簇（由权重一并计入），真正影响优化的难帧被选中，且因匹配的是聚合梯度而非追逐单点极值，不会被噪声离群帧绑架。图 4-4(d) 显示 CRAIG 选中帧对整体密度的覆盖最贴合，这正是它在右臂与两臂平均上稳定超过随机的根源。

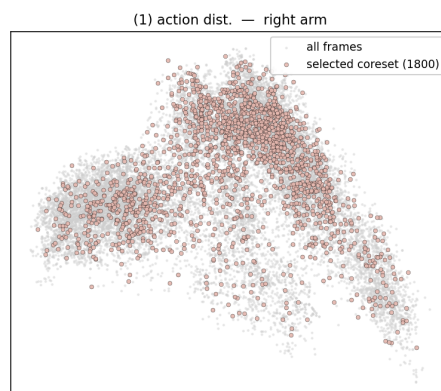
图 4-5 进一步可视化了“代表簇重加权”这一步：在梯度空间中把全池每帧划归最近的选中代表点，图 4-5(a) 中代表点按其名下帧数（代表簇质量 w_j ）着色与缩放，图 4-5(b) 给出 w_j 的分布。可以看到簇质量显著不均、呈长尾——少数代表点折叠了大量梯度冗余的易帧（ w_j 远大于 1），而难帧所在的稀疏区则由许多轻簇精细刻画；正是这一按簇质量配权重的机制，使加权子集的梯度无偏地逼近全量梯度，避免了等权选择对重尾区域的系统性低估。

综合上述机理，下面结合表 4-1 与图 4-1 的具体数值，对各方法的表现逐项印证与小结。

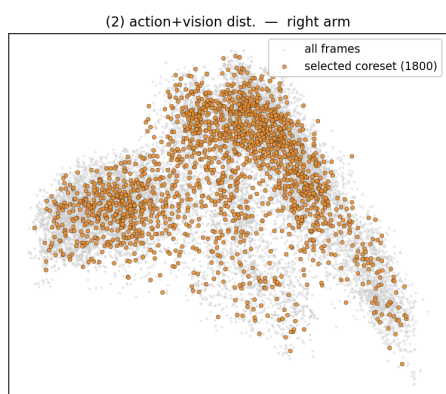
(1) 两臂平均为何用比值而非原始 MSE。右臂的动作更剧烈、MSE 量级约为左臂的 3~4 倍（表 4-1：随机左臂 3.901、右臂 12.429）。若直接对两臂原始 MSE 取算术平均，结果会被高量级的右臂主导，从而掩盖方法在左臂上的成败。为公平刻画两臂，本文的“平均”列统一采用**相对随机的比值**：每臂先除以该臂



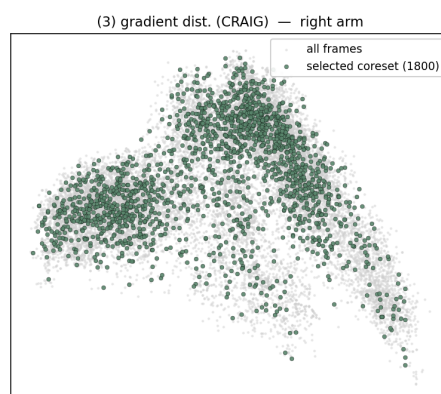
(a) 随机 (10%)



(b) 仅动作距离 (对照)

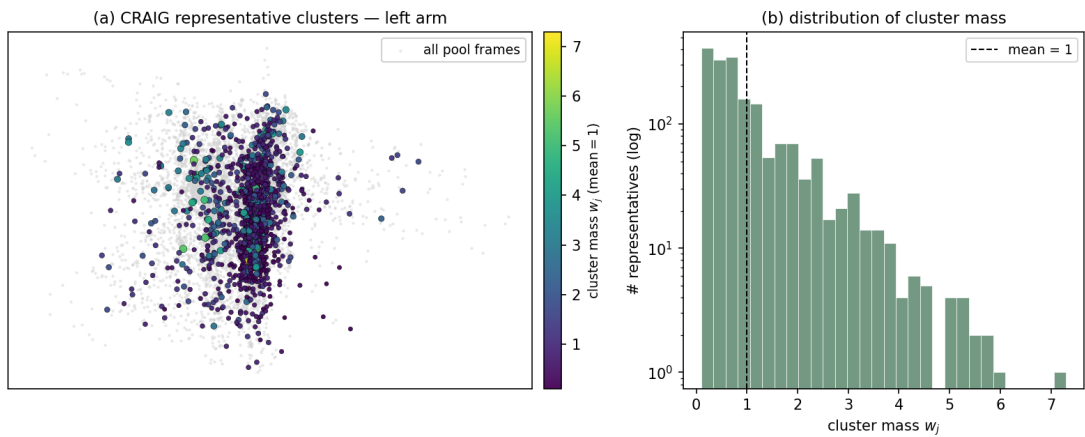


(c) 动作 + 视觉 (惊奇度)

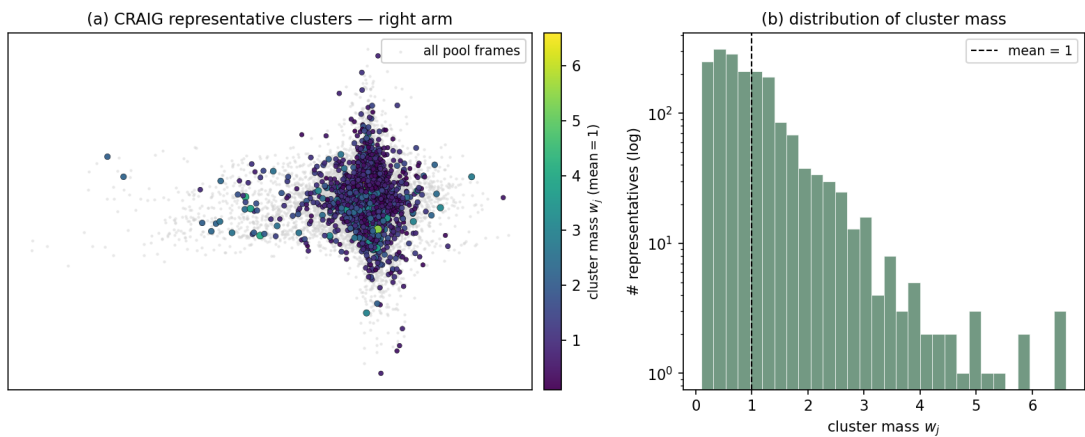


(d) 梯度距离 CRAIG

图 4-4 右臂上四种方法最终选中的 10% 核心集分布。全部 2×10^4 帧用 CLIP 特征做 2D PCA 投影 (灰点), 彩点为各方法选中的 1800 帧。CRAIG (d) 与随机 (a) 覆盖更贴合整体密度; 输入几何 (b)(c) 分布更不均。



(a) 左臂



(b) 右臂

图 4-5 CRAIG 代表簇重加权可视化（左右臂）。每行左图为梯度空间 2D PCA（灰点为全池帧，选中代表点按代表簇质量 w_j ——名下帧数、归一到均值 1——着色与缩放），右图为 w_j 的分布（纵轴对数）。两臂的簇质量均呈明显长尾：少数代表点吸收了大量梯度冗余帧，加权后子集梯度方能逼近全量；相比之下右臂的长尾更重，反映其动作更剧烈、冗余结构更不均。

随机基线（销量纲）、再对左右臂取平均，随机 = 1.000、< 1 表示优于随机。下文分析均基于这一口径。

（2）两种核心度量均优于随机，**CRAIG** 更胜一筹。在比值口径下，动作 + 视觉距离（0.984）与梯度距离 **CRAIG**（0.975）的两臂平均均跨到随机（1.000）之下，其中 **CRAIG** 最优、逼近全量（0.931）；而早期的 FPS、以及仅用单一输入通道的分量对照（仅动作 1.094、纯视觉 1.104，见第 4.4 节）则仍劣于随机——这说明动作与视觉两个通道合用才把输入几何度量推到随机之上。

（3）梯度距离 **CRAIG** 攻克右臂——唯一真正赢下重尾臂者。梯度距离在右臂上 11.569 ± 0.60 低于随机 12.429 ± 0.78 （比值 0.931，相对降低 6.9%），且两者均值互相落在对方的 $\pm 1\sigma$ 之外，差异稳健。在全部核心集方法（含动作 + 视觉及其各分量、分割净化对照）中，**CRAIG** 是唯一在重尾右臂上比值 < 1 者（其余右臂比值均 > 1，见表 4-1、表 4-2）；这也使其两臂平均比值（0.975）在表 4-1 的各方法中最优（第 4.4 节经分割净化的动作 + 视觉·抠图虽在平均上更低至 0.952，但其右臂仍劣于随机，详见下文）。

（4）输入几何度量（动作 + 视觉）只赢左臂，其平均优势全部来自左臂。动作 + 视觉在平稳的左臂上 3.387 ± 0.23 显著优于随机 3.901（比值 0.868，相对 13%），但在右臂上 13.657 劣于随机（比值 1.099，其仅动作分量更差、右臂 14.700，见第 4.4 节）。需特别指出：动作 + 视觉的两臂平均比值（0.984）虽已略优于随机，却完全由左臂的大幅领先贡献、右臂仍未赢——这恰是比值口径的价值所在：它如实暴露“平均占优 $\neq$ 右臂占优”，而原始 MSE 平均会因右臂量级主导得出相反假象。该结果印证本节前文的机理分析：输入几何偏置只在结构性强的左臂与数据需求对齐，在重尾且含噪声的右臂上与“模型真实误差”失配；加入视觉信息可缓解但不能根治。

（5）多种子验证的必要性。在单训练种子下，动作 + 视觉的右臂 MSE 曾低至 11.4、一度被误判为“同时赢两臂”；而 5 种子统计（ 13.657 ± 1.58 ）揭示那只是有利种子的假象。**CRAIG** 的领先则经得起多种子检验——这凸显了实验选择 $\text{mean} \pm \text{std}$ 对避免假阳性结论的重要性。

4.4 进一步实验：分量消融与视觉项的分割净化

第 4.3 节表明输入几何度量（动作 + 视觉）在右臂失配。为进一步厘清动作项与视觉项各自的作用、以及背景是否污染了视觉项，本节做两组进一步实验。

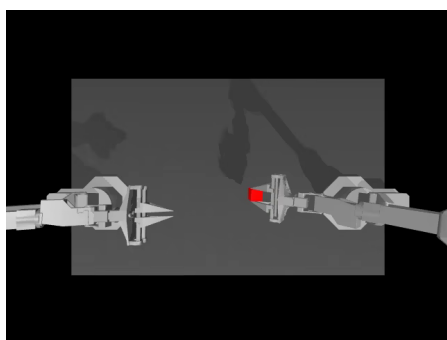
（一）分量消融。把核心度量“动作 + 视觉”拆开，单独评估仅动作距离（ $w_v=0$ ）与纯视觉距离（ $w_a=0, w_v=1$ ，距离只由视觉决定，但锚点仍为动作驱动的首/末/夹物体帧），以看清两个输入通道各自的贡献。

（二）视觉项分割净化。一个自然的疑问是：视觉项在右臂帮助有限，是否并非视觉信息无用，而是整图 CLIP 视觉距离被任务无关的背景与另一只臂的运动所污染？为检验这一猜想，本节用 SAM 3 [28] 的概念提示分割把每帧的目标臂、夹爪与被操作立方体抠出、置于纯白背景（概念提示 cube / robot arm / gripper、按检测框中心分左右；该分割代码见附录），再过同一冻结 CLIP 得到抠图视觉特征 ϕ_{seg} ，并仅用它替换式 (3.1) 中距离 M 的视觉项 d^v ；MLP 的训练与测试输入始终保持整图特征不变，故任何性能变化都能干净地归因于“选点信号是否更对齐任务”。左臂任务用左臂抠图、右臂任务用右臂抠图（分割示例见图 4-6）。视觉净化在动作 + 视觉与纯视觉两种权重档下各做“整图 ϕ vs 抠图 ϕ_{seg} ”对照。

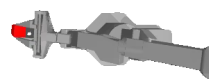
上述各对照与主实验共用同一 train/test 划分与同一组 5 训练种子，与随机基准严格配对，结果见表 4-2。图 4-6 给出 SAM 3 分割的左右臂示例。

表 4-2 核心度量“动作 + 视觉”的分量消融与视觉项分割净化（均 facility、10% 预算；左右臂为 $MSE \times 10^{-3}$ 、5 训练种子 mean $\pm$ std，平均列为两臂相对随机的 MSE 比值，口径同表 4-1、随机 = 1.000、< 1 更优）。(a) 段把“动作 + 视觉”拆为仅动作、纯视觉两个分量；(b) 段把视觉项从整图 ϕ 换为抠图 ϕ_{seg} （仅改 M 的视觉项、MLP 输入不变）。加粗为各 10% 方法中各列最优。

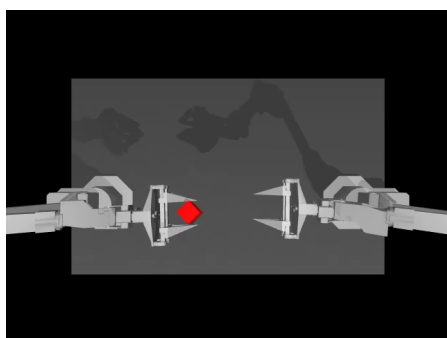
方法（均 facility、10% 预算）	左臂 MSE	右臂 MSE	平均 (比值)
随机（10%）	3.901 $\pm$ 0.42	12.429 $\pm$ 0.78	1.000
动作 + 视觉（核心度量，整图）	3.387 $\pm$ 0.23	13.657 $\pm$ 1.58	0.984
(a) 分量消融：拆解“动作 + 视觉”			
仅动作距离（ $w_v=0$ ）	3.925 $\pm$ 0.26	14.700 $\pm$ 1.84	1.094
纯视觉距离（ $w_a=0$ ，整图）	3.928 $\pm$ 0.52	14.916 $\pm$ 1.20	1.104
(b) 视觉项分割净化：整图 $\phi \rightarrow$ 抠图 ϕ_{seg}			
纯视觉 · 抠图	3.326 $\pm$ 0.13	13.354 $\pm$ 0.96	0.964
动作 + 视觉 · 抠图	3.234 $\pm$ 0.19	13.362 $\pm$ 1.02	0.952



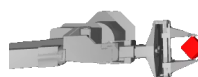
(a) 原始整图（帧 208，右臂抓取）



(b) 右臂抠图：右臂 + 夹爪 + 立方体



(c) 原始整图（帧 360，交接后左臂持物）



(d) 左臂抠图：左臂 + 夹爪 + 立方体

图 4-6 SAM 3 概念分割的左右臂示例（白底，按检测框中心分左右）。由于立方体在任一时刻只随持有它的那只臂，故每只臂取其夹持立方体的时刻采样，以完整展示“目标臂 + 夹爪 + 立方体”：上行为右臂任务——右臂抓取时刻（帧 208），右臂抠图保留右臂、夹爪及其夹持的立方体；下行为左臂任务——交接完成后（帧 360），左臂抠图保留左臂、夹爪及已接过的立方体。两行均滤除了另一只臂与背景这类对该臂动作无关、却制造视觉方差的干扰。左右臂各得一套 ϕ_{seg} ，仅用于距离 M 的视觉项。

(1) 动作与视觉两个通道缺一不可。由表 4-2(a)，把“动作 + 视觉”拆开后，仅动作（平均 1.094）与纯视觉（整图，平均 1.104）单独使用均劣于随机，且都明显差于二者合用的 0.984。这说明两个输入通道各自都不足以选出好子集——动作通道刻画姿态变化、视觉通道补足场景差异，只有加权合用才把输入几何度量推到随机之上。

(2) 抠图净化视觉项：平均反超随机、甚至全场最佳，但右臂仍未超过随机。由表 4-2(b)，仅把视觉项从整图换为抠图（白底、仅留目标臂/夹爪/立方体），左右两臂 MSE 在两种权重档下无一例外下降：纯视觉平均 1.104 $\rightarrow$ 0.964、动作 + 视觉平均 0.984 $\rightarrow$ 0.952，其中动作 + 视觉 · 抠图 (0.952) 是本文全部核心集方法中两臂平均最优者、优于 CRAIG (0.975)。降幅随视觉权重单调放大（纯视觉档左臂降 15.3%、右臂 10.5%；动作 + 视觉档因视觉项被动作项稀释，仅左 4.5%、右 2.2%），说明收益确实来自视觉项本身——背景与另一只臂的运动对整图 CLIP 视觉距离的污染真实且可观。然而这一平均优势仍主要由左臂贡献：即使用纯视觉 · 抠图，右臂比值仍 > 1 （右臂 13.354 vs 随机 12.429、劣约 7.4%），抠图未能让任何输入几何度量在重尾右臂上赢过随机。右臂的失配只部分源于背景污染，更根本的原因是其动作重尾含噪，使得任何输入几何（哪怕视觉项已被抠干净）都难以与“模型真实误差”对齐——故重尾右臂仍需梯度感知（CRAIG），二者恰好互补。

4.5 讨论与局限

笔者认为未来的核心集选择算法及实验仍有边界与改进空间：其一，实验为单任务、单 train/test 划分、固定 10% 预算，普适性有待在多任务、多划分与“预算-性能”曲线上进一步确认；其二，CRAIG 需额外训练一个 pilot 模型（约一次全量训练的开销），是其相对几何方法的代价；其三，CRAIG 在左臂未超随机，若要两臂同时严格超随机，一个自然的方向是按臂选方法（平稳左臂用几何 facility、重尾右臂用 CRAIG），两者 profile 恰好互补；其四，第 4.4 节的分量消融表明动作与视觉两个通道缺一不可（单用任一通道均劣于随机）；而用 SAM 3 分割净化视觉项的尝试表明，背景污染虽真实存在、抠图能稳定改善选点并使两臂平均比值反超随机（动作 + 视觉 · 抠图 0.952 甚至取得全场最佳平均），却仍不足以让输入几何度量在重尾右臂上超过随机，这从反面进一步印证了重尾子任

务需要任务/梯度感知信号，也提示“分割净化几何度量”与“梯度感知 CRAIG”可能互补、值得后续融合。

5 结束语

5.1 本文主要工作

本文面向 VLA 训练中的“数据冗余与算力墙”问题，借鉴人脑预测编码与网状激活系统两类认知选择机制，构建了一套核心集选择框架：以首帧/末帧/夹物体帧三类锚点对应注意力聚焦，在成对距离矩阵上做 facility-location 代表点选择并配代表簇重加权。框架的核心是帧间距离的度量方式，本文以**两种核心度量**为主线：**动作 + 视觉距离**（成对惊奇度，无监督输入几何）与**梯度距离（CRAIG）**——以 pilot 模型的末层梯度特征度量帧间距离，使加权子集的梯度逼近全量梯度。此外，本文还做了两组进一步实验：把“动作 + 视觉”拆解为仅动作、纯视觉两条对照以厘清各分量贡献，并用 SAM 3 概念分割抠出目标臂、夹爪与立方体（白底）净化距离 M 的视觉项，考察背景污染对视觉选择信号的影响。本文在 ALOHA Sim Transfer Cube 数据集上完成了从特征提取、核心集选择到 MLP 训练与多种子评估的完整实验流程，并统一采用销量纲的**相对随机比值**报告两臂平均，以避免高量级右臂主导算术平均。

5.2 主要研究结论

实验得出四点结论：其一，**梯度感知的 CRAIG 在仅用 10% 数据下，是唯一真正赢下重尾右臂（11.57 vs 随机 12.43）的度量，两臂平均比值（0.975）亦优于随机并逼近全量**，证明高质量子集确能带来更好性能；其二，纯输入几何的代表点选择只在结构性强的左臂占优、在重尾右臂上劣于随机，其两臂平均即便占优也全由左臂贡献——说明要真正解决重尾分布的任务上，选择信号须从“输入几何”转向“任务/梯度感知”；其三，分量消融显示动作与视觉两个输入通道缺一不可（单用任一通道均劣于随机、合用方超随机）；而用 SAM 3 分割净化视觉项可稳定改善选点、使输入几何度量的两臂平均反超随机（动作 + 视觉 · 抠图 0.952 取得全场最佳平均），却仍无法让其在右臂上优于随机，说明背景污染只是右臂失配的部分诱因，因此“分割净化”与“梯度感知”互补可期；其四，**两臂平均须用销量纲的比值而非原始 MSE 算术平均**（后者被高量级右臂主导），且多训练种子（mean±std）评估对得出可靠结论至关重要，二者共同揭穿了单

种子、原始平均下“几何方法两臂全赢”的假象，避免了假阳性结论。

5.3 认识与体会

这次课设让我体会最深的，是在小数据集上把核心集选择算法做好其实是比较不容易的。方法的演进，几乎是被一个个负面的实验结果逼出来的。我最初用最远点采样 (FPS)，但它在右臂上始终打不过随机基准——动作 + 视觉距离下右臂 MSE 高达 17.0、仅动作距离也有 13.7，均明显劣于随机的 12.4（见表 4-1）。对方法进行复盘后我意识到：FPS 近似被离群点主导，把预算耗在了噪声帧上，于是改用近似 k-median、贴合分布的 facility；但随之而来的也并不是 facility 就在左右臂的平均结果上就都比随机的效果好。经历这两次的实验后，进一步顺着“冗余应按一帧对模型的边际贡献来定义”这一思路转入梯度空间后，CRAIG 才把右臂做到 11.57 ± 0.60 、低于随机 12.43 ± 0.78 （相对降低 6.9%），两臂平均比值 0.975 也压到随机（1.0）之下、逼近全量（0.931）。但该方法也并没有在双臂上均优于随机（左臂未超随机），因此后续我又用 SAM 3 剔除背景等噪声，想看能不能让双臂都胜过随机，但从实验结果看，右臂上依旧弱于随机。可见想在双臂上都做到优于随机还是挺不容易的。这中间还有个容易踩的坑：左右臂 MSE 量级差了 3~4 倍，一开始我直接对原始 MSE 取平均，结果几乎被右臂主导、把左臂的成败都淹没了；后来改用每臂先对随机基线归一化再平均的比值口径，才得到对两臂公平、也更能反映方法真实优劣的结论。

但本次实验也实际锻炼了我独立思考问题和解决问题的能力，处于对具身智能的兴趣和以往的研究经历，我选择了这个课题，相信这以后对我未来具身方向的研究也是有帮助的，比如具身的自监督学习，怎么构造高质量且无冗余的数据集是一个关键挑战，或许本次课设实验中对核心集算法的思考就能对我今后做这方面的科研研究奠定比较好的基础，以及对这个问题做好更深的认识。

主要模块代码

下面按运行先后顺序给出本实验的全部核心代码，并提供核心注释、可直接阅读。

config.py —— 实验参数集中配置

```
1  """所有可调参数集中放这里。带 _env_* 的项可被同名环境变量覆盖，方便用脚本批量扫参。"""
2  import os
3
4
5  def _env_int(name, default):
6      v = os.environ.get(name)
7      return default if v is None else int(v)
8
9
10 def _env_str(name, default):
11     return os.environ.get(name, default)
12
13
14 def _env_bool(name, default):
15     v = os.environ.get(name)
16     return default if v is None else v.strip().lower() in ("1", "true", "yes", "on")
17
18
19 def _env_float(name, default):
20     v = os.environ.get(name)
21     return default if v is None else float(v)
22
23 # ----- 路径 -----
24 PROJECT_DIR = r"E:\vision-roc"
25 CACHE_DIR = os.path.join(PROJECT_DIR, ".cache")
26 FEATURE_PATH = os.path.join(PROJECT_DIR, "features", "clip_features.npz")
27 # SAM3 抠图(白底)的 CLIP 视觉特征 phi_seg, 由 extract_seg_features.py 生成; 左右臂各一份
28 SEG_FEATURE_PATH = os.path.join(PROJECT_DIR, "features", "seg_features.npz")
29 SEG_IMG_DIR = os.path.join(PROJECT_DIR, "aloha_sim_transfer_cube_human_seg")
30 # 所有实验输出(结果 JSON / 抓取帧索引 / 可视化)都放这里, 每个 task_id 一个子目录
31 OUTPUT_DIR = r"E:\vision-roc\expiement-output"
32
33 # ----- 数据集 / 模型 -----
34 REPO_ID = "lerobot/aloha_sim_transfer_cube_human"
35 CLIP_NAME = "openai/clip-vit-base-patch32" # 图像和文本都用它, 各输出 512 维
36 INSTRUCTION = "Pick up the cube with the right arm and transfer it to the left arm."
37
38 # ----- 实验协议 -----
39 SEED = 42 # 划分 train/test 用的随机种子
```

```

40 N_TEST_EP      = 5          # 50 条 episode -> 5 条测试 / 45 条训练
41 BUDGET_RATIO = 0.10        # 核心集预算 = 训练帧的 10%
42 ARM           = "both"     # "both" | "left" | "right" | "auto"(按方差自动选臂)
43
44 # ----- 距离里动作/视觉的权重 -----
45 # 度量二(动作+视觉): W_ACT=0.7, W_VIS=0.3 ; 纯视觉对照: W_ACT=0, W_VIS=1 。均可用环境变量覆盖。
46 W_ACT = _env_float("W_ACT", 0.7)
47 W_VIS = _env_float("W_VIS", 0.3)
48 EPS   = 1e-8
49
50 # 距离矩阵 M 的视觉项用哪份 CLIP 特征:
51 #   "full" = 整图 phi(clip_features.npz) ; "seg" = SAM3 抠图 phi_seg(seg_features.npz, 左右臂分别)
52 # 仅影响 M 的视觉项(即选点信号), MLP 的训练/测试输入始终用整图特征, 不受此开关影响。
53 VIS_FEATURE_SRC = _env_str("VIS_FEATURE_SRC", "full")
54
55 # 两臂"比值平均"用的共享随机基线(每臂 random 的 5 种子均值; SEED=42 划分 + 种子[0..4] 全实验一致)。
56 # 平均改算 ratio_arm = 方法_arm / random_arm 再对两臂取平均, 消掉左右臂 MSE 量级差。
57 # 当本次跑出了 random(RUN_BASELINES=True)时优先用本次的; RUN_BASELINES=False 时回退到这条参照。
58 REF_RANDOM = {"left": 0.0039012286346405745, "right": 0.012428723834455013}
59
60 # ----- 动作表征(影响距离里 ||a_i-a_j|| 怎么算) -----
61 STANDARDIZE_ACT = _env_bool("STANDARDIZE_ACT", True) # 算距离前对动作逐维 z-score, 消量纲
62 USE_DYNAMICS    = _env_bool("USE_DYNAMICS", True)   # 把速度/加速度也拼进动作表征
63 W_POS = 1.0    # 位置/速度/加速度三块在距离里的相对权重(都已各自 z-score)
64 W_VEL = 1.0
65 W_ACC = 0.5
66
67 # ----- 核心集选择 -----
68 SURPRISE_MODE = _env_int("SURPRISE_MODE", 1)        # 1 = 动作+视觉距离; 2 = 只看动作
69 GRASP_THR     = 0.5                                # 夹物体帧检测的开/闭分界(占幅度的比例)
70 Q_PERCENTILE  = 0.10                                # 锚点去重阈值取距离的这个分位数
71 CORESET_METHOD = _env_str("CORESET_METHOD", "facility") # fps=最远点采样; facility=代表点选择;
    craig=梯度空间代表点+样本加权
72 SIGMA_Q       = 0.5                                # facility/CRAIG 相似度带宽, 取距离的这个分
    位数
73 ADAPTIVE_BUDGET = _env_bool("ADAPTIVE_BUDGET", False) # 重要度加权 facility (高信息帧多分代表点),
    仅 facility 生效
74 REWEIGHT      = _env_bool("REWEIGHT", False)        # 给选中点配 CRAIG 权重做加权训练
75 M_PATH        = os.path.join(CACHE_DIR, "pairwise_M.dat") # 距离矩阵 memmap 路径(每只臂各建一份)
76 M_BLOCK       = 1000                                # 构建距离矩阵的分块行数
77
78 # 全量/随机基线只跟监督方式有关, 和 SURPRISE_MODE/CORESET_METHOD 无关; 批量扫参时置 False 可跳过、不重
    复跑
79 RUN_BASELINES = _env_bool("RUN_BASELINES", True)
80 RUN_FULL      = _env_bool("RUN_FULL", True)          # 在 RUN_BASELINES 之内再单独控制要不要跑全量
81
82 # ----- MLP -----
83 EVAL_ORIGINAL = _env_bool("EVAL_ORIGINAL", True)     # True=在原始动作空间算 MSE; False=在归一化

```

```

    空间算
84 SUPERVISE_ORIGINAL = _env_bool("SUPERVISE_ORIGINAL", True) # True=用原始动作监督训练; False=用归一
    化动作监督
85 HIDDEN          = (256, 128)
86 DROPOUT          = 0.1
87 LR               = 1e-3
88 WEIGHT_DECAY     = 1e-4
89 BATCH_SIZE       = 64
90 EPOCHS           = 200
91 RANDOM_SEEDS     = [0, 1, 2, 3, 4] # 随机基准与各方法都用这几个种子, 取 mean±std

```

extract_features.py ——用冻结的 CLIP 离线提取图文特征与动作标签

```

1  """
2  一次性离线提取 CLIP 特征(图像 + 文本)与单臂动作标签, 存成 npz。
3  产出: FEATURE_PATH 内含
4      img_feat [N,512] float32  CLIP 图像特征(observation.images.top)
5      txt_feat [512]  float32  CLIP 文本特征(常量指令, 全帧共用)
6      action   [N,14] float32  原始 14-DOF 动作(双臂)
7      ep_id    [N]    int      episode 编号
8      frame_id [N]    int      episode 内帧序号
9  运行: python extract_features.py
10 """
11 import sys
12 sys.path.insert(0, r"E:\vision-roc\.pylibs")
13 import os
14 import config as cfg
15
16 # 缓存设置(必须在 import transformers/torch 之前设置)
17 os.environ.setdefault("HF_HOME", os.path.join(cfg.CACHE_DIR, "huggingface"))
18 os.environ.setdefault("HF_DATASETS_CACHE", os.path.join(cfg.CACHE_DIR, "huggingface", "datasets"))
19 os.environ.setdefault("TORCH_HOME", os.path.join(cfg.CACHE_DIR, "torch"))
20
21 import numpy as np
22 import torch
23 from PIL import Image
24 from transformers import CLIPModel, CLIPProcessor
25
26 # lerobot 版本间 import 路径不一, 做好兼容
27 try:
28     from lerobot.common.datasets.lerobot_dataset import LeRobotDataset
29 except Exception:
30     from lerobot.datasets.lerobot_dataset import LeRobotDataset
31
32
33 def main():
34     device = "cuda" if torch.cuda.is_available() else "cpu"
35     print(f"[device] {device}")

```

```

36
37 ds = LeRobotDataset(cfg.REPO_ID)
38 print(f"[dataset] {cfg.REPO_ID} frames={len(ds)}")
39
40 model = CLIPModel.from_pretrained(cfg.CLIP_NAME).to(device).eval()
41 processor = CLIPProcessor.from_pretrained(cfg.CLIP_NAME)
42
43 # ---- 文本特征(常量, 只算一次)----
44 with torch.no_grad():
45     tin = processor(text=[cfg.INSTRUCTION], return_tensors="pt", padding=True).to(device)
46     tin_out = model.text_model(**tin)
47     txt_embeds = model.text_projection(tin_out.pooler_output)
48     txt_feat = txt_embeds.cpu().numpy()[0].astype(np.float32) # [512]
49
50 # ---- 图像特征 + 动作 + 索引 ----
51 loader = torch.utils.data.DataLoader(ds, batch_size=64, shuffle=False, num_workers=0)
52 img_feats, actions, ep_ids, frame_ids = [], [], [], []
53
54 for bi, batch in enumerate(loader):
55     imgs = batch["observation.images.top"] # [B,3,H,W], float[0,1]
56     imgs_np = (imgs.permute(0, 2, 3, 1).numpy() * 255).astype(np.uint8) # [B,H,W,3]
57     pil = [Image.fromarray(a) for a in imgs_np]
58     pin = processor(images=pil, return_tensors="pt").to(device)
59     with torch.no_grad():
60         pin_out = model.vision_model(**pin)
61         img_embeds = model.visual_projection(pin_out.pooler_output)
62         feat = img_embeds.cpu().numpy().astype(np.float32)
63         img_feats.append(feat)
64         actions.append(batch["action"].numpy().astype(np.float32))
65         ep_ids.append(batch["episode_index"].numpy().astype(np.int64))
66         frame_ids.append(batch["frame_index"].numpy().astype(np.int64))
67         if bi % 20 == 0:
68             print(f" batch {bi}/{len(loader)}")
69
70 img_feat = np.concatenate(img_feats, 0)
71 action = np.concatenate(actions, 0)
72 ep_id = np.concatenate(ep_ids, 0)
73 frame_id = np.concatenate(frame_ids, 0)
74
75 os.makedirs(os.path.dirname(cfg.FEATURE_PATH), exist_ok=True)
76 np.savez(cfg.FEATURE_PATH,
77         img_feat=img_feat, txt_feat=txt_feat,
78         action=action, ep_id=ep_id, frame_id=frame_id)
79 print(f"[saved] {cfg.FEATURE_PATH}")
80 print(f" img_feat {img_feat.shape} txt_feat {txt_feat.shape} action {action.shape}")
81
82
83 if __name__ == "__main__":

```

```
84     main()
```

extract_seg_features.py ——对 SAM 3 抠图 (白底) 的左右臂分别提 CLIP 视觉特征 ϕ_{seg}

```
1  """
2  对 SAM3 抠图(白底:目标臂+夹爪+立方体)的左右臂图像分别提 CLIP 视觉特征, 存成 npz。
3  和 extract_features.py 用同一个 CLIP(openai/clip-vit-base-patch32)、同样的预处理,
4  保证抠图特征 phi_seg 与整图特征 phi 唯一的差别就是输入图像本身。
5
6  输入: 分割目录   aloha_sim_transfer_cube_human_seg/{left,right}/{index:06d}.png
7         其中 index == clip_features.npz 里的行号(已用 manifest 校验过 0 处错位)。
8  产出: SEG_FEATURE_PATH 内含
9         img_feat_left  [N,512] float32   左臂抠图 CLIP 视觉特征
10        img_feat_right [N,512] float32   右臂抠图 CLIP 视觉特征
11        ep_id          [N]      int      episode 编号(从 clip_features.npz 拷贝, 供对齐核对)
12        frame_id       [N]      int      episode 内帧序号
13  注意: 本脚本只产视觉特征 phi_seg, 不动 clip_features.npz, 也不碰文本/动作。
14  运行: python extract_seg_features.py
15  """
16  import sys
17  sys.path.insert(0, r"E:\vision-roc\pylibs")
18  import os
19  import config as cfg
20
21  # 缓存设置(必须在 import transformers/torch 之前设置, 一律落非 C 盘)
22  os.environ.setdefault("HF_HOME", os.path.join(cfg.CACHE_DIR, "huggingface"))
23  os.environ.setdefault("HF_DATASETS_CACHE", os.path.join(cfg.CACHE_DIR, "huggingface", "datasets"))
24  os.environ.setdefault("TORCH_HOME", os.path.join(cfg.CACHE_DIR, "torch"))
25
26  import numpy as np
27  import torch
28  from PIL import Image
29  from transformers import CLIPModel, CLIPProcessor
30
31
32  def _encode_side(model, processor, device, side, n, seg_dir, batch=64):
33      """按 index 顺序读取一侧抠图, 批量过 CLIP 视觉塔, 返回 [n,512] float32。"""
34      feats = []
35      buf_imgs, buf_idx = [], []
36
37      def _flush():
38          if not buf_imgs:
39              return
40          pin = processor(images=buf_imgs, return_tensors="pt").to(device)
41          with torch.no_grad():
42              out = model.vision_model(**pin)
```

```

43         emb = model.visual_projection(out.pooler_output)
44         feats.append(emb.cpu().numpy().astype(np.float32))
45         buf_imgs.clear()
46         buf_idx.clear()
47
48     for i in range(n):
49         fp = os.path.join(seg_dir, side, f"{i:06d}.png")
50         if not os.path.exists(fp):
51             raise FileNotFoundError(f"缺少抠图: {fp}")
52         buf_imgs.append(Image.open(fp).convert("RGB"))
53         buf_idx.append(i)
54         if len(buf_imgs) == batch:
55             _flush()
56             if (i // batch) % 20 == 0:
57                 print(f" [{side}] {i+1}/{n}")
58         _flush()
59     return np.concatenate(feats, 0)
60
61
62 def main():
63     device = "cuda" if torch.cuda.is_available() else "cpu"
64     print(f"[device] {device}")
65
66     seg_dir = cfg.SEG_IMG_DIR
67     assert os.path.isdir(seg_dir), f"找不到分割目录: {seg_dir}"
68
69     # 用 clip_features.npz 的长度与 ep/frame 索引做对齐基准
70     d = np.load(cfg.FEATURE_PATH)
71     n = int(d["img_feat"].shape[0])
72     ep_id = d["ep_id"].astype(np.int64)
73     frame_id = d["frame_id"].astype(np.int64)
74     print(f"[align] N={n} (来自 {os.path.basename(cfg.FEATURE_PATH)})")
75
76     model = CLIPModel.from_pretrained(cfg.CLIP_NAME).to(device).eval()
77     processor = CLIPProcessor.from_pretrained(cfg.CLIP_NAME)
78
79     print("[encode] left ...")
80     feat_left = _encode_side(model, processor, device, "left", n, seg_dir)
81     print("[encode] right ...")
82     feat_right = _encode_side(model, processor, device, "right", n, seg_dir)
83
84     os.makedirs(os.path.dirname(cfg.SEG_FEATURE_PATH), exist_ok=True)
85     np.savez(cfg.SEG_FEATURE_PATH,
86             img_feat_left=feat_left, img_feat_right=feat_right,
87             ep_id=ep_id, frame_id=frame_id)
88     print(f"[saved] {cfg.SEG_FEATURE_PATH}")
89     print(f" img_feat_left {feat_left.shape} img_feat_right {feat_right.shape}")
90

```

```
91
92 if __name__ == "__main__":
93     main()
```

SAM 3 分割代码

需要特别说明的是，出于 sam3 在 cpu 上跑 20000 次推理的时间影响，第 4.4 节的分割并非在笔者自己的本地电脑上运行，而是在具有算力服务器上完成：以概念提示 cube / robot arm / gripper 做开放词表实例分割，按检测框中心落在画面左/右半幅把两条手臂分开，将“目标臂 + 夹爪 + 立方体”合成到纯白背景，输出左右臂各一套 PNG（即上文 extract_seg_features.py 的输入）。分割置信度阈值 0.3。以下为该套服务器端代码。

segment_aloha_sam3.py —— SAM 3 分割主程序（概念提示 + 左右分割 + 白底合成 + manifest）

```
1  #!/usr/bin/env python3
2  """
3  Offline SAM3 segmentation for lerobot/aloha_sim_transfer_cube_human.
4
5  Per frame, keeps target arm + gripper + cube (by text prompt + left/right split),
6  sets background to white. Writes left/ and right/ PNG trees for later CLIP phi_seg.
7
8  Example (run after sourcing Ascend + conda activate sam3):
9
10     source /usr/local/Ascend/cann-8.5.2/set_env.sh
11     conda activate sam3
12     python /mnt/yyli/scripts/segment_aloha_sam3.py --npu-id 2
13
14  Resume: existing PNGs are skipped unless --overwrite.
15  """
16
17  from __future__ import annotations
18
19  import argparse
20  import json
21  import os
22  import time
23  from pathlib import Path
24
25  import av
26  import numpy as np
27  import pandas as pd
28  import torch
```

```

29 from PIL import Image
30 from tqdm import tqdm
31
32 # Pin to one NPU before torch_npu initializes visible devices.
33 def _parse_args_early() -> argparse.Namespace:
34     parser = argparse.ArgumentParser(add_help=False)
35     parser.add_argument("--npu-id", type=int, default=2)
36     args, _ = parser.parse_known_args()
37     return args
38
39
40 _EARLY = _parse_args_early()
41 os.environ.setdefault("ASCEND_RT_VISIBLE_DEVICES", str(_EARLY.npu_id))
42
43 import torch_npu # noqa: E402
44
45 from sam3.model_builder import build_sam3_image_model # noqa: E402
46 from sam3.model.sam3_image_processor import Sam3Processor # noqa: E402
47
48
49 TEXT_PROMPTS = ("cube", "robot arm", "gripper")
50 SIDE_PROMPTS = ("robot arm", "gripper")
51
52
53 def load_frame_index_table(dataset_root: Path) -> pd.DataFrame:
54     chunks = sorted((dataset_root / "data" / "chunk-000").glob("*.parquet"))
55     if not chunks:
56         raise FileNotFoundError(f"No parquet under {dataset_root / 'data'}")
57     df = pd.concat([pd.read_parquet(p) for p in chunks], ignore_index=True)
58     df = df.sort_values("index").reset_index(drop=True)
59     required = {"index", "episode_index", "frame_index"}
60     missing = required - set(df.columns)
61     if missing:
62         raise KeyError(f"Missing columns in parquet: {missing}")
63     return df
64
65
66 def decode_video_frame(video_path: Path, frame_idx: int) -> Image.Image:
67     container = av.open(str(video_path))
68     stream = container.streams.video[0]
69     for i, frame in enumerate(container.decode(stream)):
70         if i == frame_idx:
71             return Image.fromarray(frame.to_ndarray(format="rgb24"))
72     raise IndexError(f"Frame {frame_idx} not found in {video_path}")
73
74
75 class SequentialVideoReader:
76     """Forward-only decoder; assumes frame indices are mostly non-decreasing."""

```

```

77
78     def __init__(self, video_path: Path):
79         self.video_path = video_path
80         self._container = av.open(str(video_path))
81         self._stream = self._container.streams.video[0]
82         self._cursor = 0
83         self._decoder = self._container.decode(self._stream)
84
85     def read(self, frame_idx: int) -> Image.Image:
86         if frame_idx < self._cursor:
87             self._rewind()
88         while self._cursor <= frame_idx:
89             try:
90                 frame = next(self._decoder)
91             except StopIteration as exc:
92                 raise IndexError(
93                     f"Frame {frame_idx} not found in {self.video_path} (cursor={self._cursor})"
94                 ) from exc
95             if self._cursor == frame_idx:
96                 self._cursor += 1
97                 return Image.fromarray(frame.to_ndarray(format="rgb24"))
98             self._cursor += 1
99         raise IndexError(f"Frame {frame_idx} not found in {self.video_path}")
100
101     def _rewind(self) -> None:
102         self._container.close()
103         self._container = av.open(str(self.video_path))
104         self._stream = self._container.streams.video[0]
105         self._cursor = 0
106         self._decoder = self._container.decode(self._stream)
107
108     def close(self) -> None:
109         self._container.close()
110
111
112     def _empty_mask(h: int, w: int) -> torch.Tensor:
113         return torch.zeros(h, w, dtype=torch.bool)
114
115
116     def _side_mask(
117         masks: torch.Tensor,
118         boxes: torch.Tensor,
119         side: str,
120         img_w: int,
121         img_h: int,
122     ) -> torch.Tensor:
123         """Union masks whose box center falls on the chosen half of the image."""
124         combined = _empty_mask(img_h, img_w)

```

```

125     if masks.numel() == 0:
126         return combined
127
128     mid = img_w / 2.0
129     for i in range(masks.shape[0]):
130         cx = (boxes[i, 0] + boxes[i, 2]).item() / 2.0
131         on_left = cx < mid
132         if (side == "left" and on_left) or (side == "right" and not on_left):
133             combined |= masks[i, 0].bool().cpu()
134     return combined
135
136
137 def _union_all_masks(masks: torch.Tensor, img_h: int, img_w: int) -> torch.Tensor:
138     combined = _empty_mask(img_h, img_w)
139     if masks.numel() == 0:
140         return combined
141     for i in range(masks.shape[0]):
142         combined |= masks[i, 0].bool().cpu()
143     return combined
144
145
146 def collect_prompt_masks(
147     processor: Sam3Processor,
148     state: dict,
149     prompt: str,
150 ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
151     processor.reset_all_prompts(state)
152     out = processor.set_text_prompt(prompt=prompt, state=state)
153     masks = out.get("masks", torch.empty(0))
154     boxes = out.get("boxes", torch.empty(0))
155     scores = out.get("scores", torch.empty(0))
156     if masks.numel() == 0:
157         return masks, boxes, scores
158     return masks.cpu(), boxes.cpu(), scores.cpu()
159
160
161 def build_arm_mask(
162     prompt_results: dict[str, tuple[torch.Tensor, torch.Tensor, torch.Tensor]],
163     side: str,
164     img_w: int,
165     img_h: int,
166 ) -> torch.Tensor:
167     cube_masks, _, _ = prompt_results["cube"]
168     combined = _union_all_masks(cube_masks, img_h, img_w)
169
170     for prompt in SIDE_PROMPTS:
171         masks, boxes, _ = prompt_results[prompt]
172         combined |= _side_mask(masks, boxes, side, img_w, img_h)

```

```

173
174     return combined
175
176
177 def apply_white_background(image: Image.Image, mask: torch.Tensor) -> Image.Image:
178     rgb = np.array(image, dtype=np.uint8)
179     if mask.numel() == 0 or not mask.any():
180         return Image.fromarray(rgb)
181
182     mask_np = mask.numpy()
183     out = np.full_like(rgb, 255)
184     out[mask_np] = rgb[mask_np]
185     return Image.fromarray(out)
186
187
188 def write_progress(
189     progress_path: Path,
190     *,
191     npu_id: int,
192     start: int,
193     end: int,
194     loop_i: int,
195     global_idx: int | None,
196     processed: int,
197     skipped: int,
198     failed: int,
199     status: str,
200     t_start: float,
201 ) -> None:
202     total = end - start
203     done = loop_i - start + 1
204     elapsed = max(time.time() - t_start, 1e-6)
205     rate = done / elapsed
206     remaining = max(total - done, 0)
207     eta_sec = remaining / rate if rate > 0 else None
208     payload = {
209         "npu_id": npu_id,
210         "start_index": start,
211         "end_index": end,
212         "total": total,
213         "done": done,
214         "processed": processed,
215         "skipped": skipped,
216         "failed": failed,
217         "pct": round(100.0 * done / total, 2) if total else 100.0,
218         "current_global_index": global_idx,
219         "status": status,
220         "elapsed_sec": round(elapsed, 1),

```

```

221         "eta_sec": round(eta_sec, 1) if eta_sec is not None else None,
222         "frames_per_sec": round(rate, 3),
223         "updated_at": time.strftime("%Y-%m-%d %H:%M:%S"),
224     }
225     tmp = progress_path.with_suffix(".json.tmp")
226     tmp.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
227     tmp.replace(progress_path)
228
229
230 def parse_args() -> argparse.Namespace:
231     parser = argparse.ArgumentParser(description="SAM3 segmentation for ALOHA transfer cube dataset")
232     parser.add_argument(
233         "--dataset-root",
234         type=Path,
235         default=Path("/mnt/yyli/data/aloha_sim_transfer_cube_human"),
236     )
237     parser.add_argument(
238         "--checkpoint",
239         type=Path,
240         default=Path("/mnt/sam3/sam3/sam3.pt"),
241     )
242     parser.add_argument(
243         "--output-root",
244         type=Path,
245         default=Path("/mnt/yyli/data/aloha_sim_transfer_cube_human_seg"),
246     )
247     parser.add_argument("--npu-id", type=int, default=2, help="Physical NPU id (via ASCEND_RT_VISIBLE_DEVICES)")
248     parser.add_argument("--confidence", type=float, default=0.3, help="SAM3 mask confidence threshold")
249     parser.add_argument("--start-index", type=int, default=0, help="Global frame index (inclusive)")
250     parser.add_argument("--end-index", type=int, default=-1, help="Global frame index (exclusive), -1 = all")
251     parser.add_argument("--overwrite", action="store_true", help="Re-segment even if PNG exists")
252     parser.add_argument("--max-frames", type=int, default=-1, help="Limit frames processed (debug)")
253     return parser.parse_args()
254
255
256 def main() -> None:
257     args = parse_args()
258     device = "npu:0" # after ASCEND_RT_VISIBLE_DEVICES, logical device 0 == physical npu-id
259
260     dataset_root = args.dataset_root.expanduser().resolve()
261     output_root = args.output_root.expanduser().resolve()
262     left_dir = output_root / "left"

```

```

263     right_dir = output_root / "right"
264     meta_dir = output_root / "meta"
265     for d in (left_dir, right_dir, meta_dir):
266         d.mkdir(parents=True, exist_ok=True)
267
268     info = json.loads((dataset_root / "meta" / "info.json").read_text())
269     video_key = "observation.images.top"
270     video_rel = info["video_path"].format(
271         video_key=video_key,
272         chunk_index=0,
273         file_index=0,
274     )
275     video_path = dataset_root / video_rel
276     if not video_path.is_file():
277         raise FileNotFoundError(video_path)
278     if not args.checkpoint.is_file():
279         raise FileNotFoundError(args.checkpoint)
280
281     df = load_frame_index_table(dataset_root)
282     total = len(df)
283     end = total if args.end_index < 0 else min(args.end_index, total)
284     start = max(0, args.start_index)
285     if args.max_frames > 0:
286         end = min(end, start + args.max_frames)
287
288     manifest_path = meta_dir / f"manifest_npu{args.npu_id}.jsonl"
289     manifest_mode = "a" if manifest_path.exists() else "w"
290     progress_path = meta_dir / f"progress_npu{args.npu_id}.json"
291
292     write_progress(
293         progress_path,
294         npu_id=args.npu_id,
295         start=start,
296         end=end,
297         loop_i=start - 1,
298         global_idx=None,
299         processed=0,
300         skipped=0,
301         failed=0,
302         status="loading_model",
303         t_start=time.time(),
304     )
305
306     print(f"Dataset: {dataset_root}")
307     print(f"Video: {video_path}")
308     print(f"Checkpoint: {args.checkpoint}")
309     print(f"Output: {output_root}")
310     print(f"NPU: physical id {args.npu_id} (ASCEND_RT_VISIBLE_DEVICES={os.environ.get('

```

```

        ASCEND_RT_VISIBLE_DEVICES'}}))")
311 print(f"Frames:  [{start}, {end}] / {total}")
312
313 t0 = time.time()
314 model = build_sam3_image_model(
315     checkpoint_path=str(args.checkpoint),
316     load_from_HF=False,
317     device=device,
318 )
319 processor = Sam3Processor(model, device=device, confidence_threshold=args.confidence)
320 print(f"Model loaded in {time.time() - t0:.1f}s on {device}")
321
322 job_start = time.time()
323 write_progress(
324     progress_path,
325     npu_id=args.npu_id,
326     start=start,
327     end=end,
328     loop_i=start - 1,
329     global_idx=None,
330     processed=0,
331     skipped=0,
332     failed=0,
333     status="running",
334     t_start=job_start,
335 )
336
337 processed = skipped = failed = 0
338 reader = SequentialVideoReader(video_path)
339 try:
340     with manifest_path.open(manifest_mode, encoding="utf-8") as manifest_f:
341         for i in tqdm(range(start, end), total=end - start):
342             r = df.iloc[i]
343             global_idx = int(r["index"])
344             ep_idx = int(r["episode_index"])
345             fr_idx = int(r["frame_index"])
346             left_path = left_dir / f"{global_idx:06d}.png"
347             right_path = right_dir / f"{global_idx:06d}.png"
348
349             try:
350                 if not args.overwrite and left_path.is_file() and right_path.is_file():
351                     skipped += 1
352             else:
353                 try:
354                     image = reader.read(global_idx)
355                     img_w, img_h = image.width, image.height
356
357                     state = processor.set_image(image)

```

```

358         prompt_results = {
359             prompt: collect_prompt_masks(processor, state, prompt)
360             for prompt in TEXT_PROMPTS
361         }
362
363         left_mask = build_arm_mask(prompt_results, "left", img_w, img_h)
364         right_mask = build_arm_mask(prompt_results, "right", img_w, img_h)
365
366         left_img = apply_white_background(image, left_mask)
367         right_img = apply_white_background(image, right_mask)
368         left_img.save(left_path)
369         right_img.save(right_path)
370
371         record = {
372             "index": global_idx,
373             "episode_index": ep_idx,
374             "frame_index": fr_idx,
375             "left_path": str(left_path.relative_to(output_root)),
376             "right_path": str(right_path.relative_to(output_root)),
377             "left_mask_pixels": int(left_mask.sum().item()) if left_mask.numel
378             () else 0,
379             "right_mask_pixels": int(right_mask.sum().item()) if right_mask.
380             numel() else 0,
381             "ok": True,
382         }
383         processed += 1
384     except Exception as exc: # noqa: BLE001 — log and continue batch job
385         failed += 1
386         record = {
387             "index": global_idx,
388             "episode_index": ep_idx,
389             "frame_index": fr_idx,
390             "ok": False,
391             "error": repr(exc),
392         }
393
394     manifest_f.write(json.dumps(record, ensure_ascii=False) + "\n")
395     manifest_f.flush()
396 finally:
397     write_progress(
398         progress_path,
399         npu_id=args.npu_id,
400         start=start,
401         end=end,
402         loop_i=i,
403         global_idx=global_idx,
404         processed=processed,
405         skipped=skipped,

```

```

404             failed=failed,
405             status="running",
406             t_start=job_start,
407         )
408     finally:
409         reader.close()
410
411     write_progress(
412         progress_path,
413         npu_id=args.npu_id,
414         start=start,
415         end=end,
416         loop_i=end - 1,
417         global_idx=None,
418         processed=processed,
419         skipped=skipped,
420         failed=failed,
421         status="done",
422         t_start=job_start,
423     )
424
425     summary = {
426         "processed": processed,
427         "skipped": skipped,
428         "failed": failed,
429         "start_index": start,
430         "end_index": end,
431         "confidence": args.confidence,
432         "checkpoint": str(args.checkpoint),
433         "npu_id": args.npu_id,
434     }
435     (meta_dir / f"run_summary_npu{args.npu_id}.json").write_text(
436         json.dumps(summary, indent=2), encoding="utf-8"
437     )
438     print(json.dumps(summary, indent=2))
439
440
441 if __name__ == "__main__":
442     main()

```

run_segment_6npu.sh —— 分片并行启动脚本

```

1  #!/usr/bin/env bash
2  # Launch 6-way SAM3 segmentation on NPU 2-7 and optionally watch progress.
3  set -euo pipefail
4
5  WATCH=0
6  EXTRA_ARGS=()

```

```

7  for arg in "$@"; do
8      if [[ "$arg" == "--watch" ]]; then
9          WATCH=1
10         else
11             EXTRA_ARGS+=("$arg")
12         fi
13     done
14
15     source /usr/local/Ascend/cann-8.5.2/set_env.sh
16     source /home/yyli/miniconda3/etc/profile.d/conda.sh
17     conda activate sam3
18
19     python -c "import decorator" 2>/dev/null || pip install -q decorator attrs scipy absl-py
20         cloudpickle ml-dtypes tornado psutil
21
22     SCRIPT=/mnt/yyli/scripts/segment_aloha_sam3.py
23     DATA=/mnt/yyli/data/aloha_sim_transfer_cube_human
24     OUT=/mnt/yyli/data/aloha_sim_transfer_cube_human_seg
25     CKPT=/mnt/sam3/sam3/sam3.pt
26     LOGDIR=/mnt/yyli/logs
27     mkdir -p "$LOGDIR" "$OUT/meta"
28
29     launch() {
30         local npu=$1 start=$2 end=$3
31         nohup python "$SCRIPT" \
32             --npu-id "$npu" \
33             --start-index "$start" \
34             --end-index "$end" \
35             --checkpoint "$CKPT" \
36             --dataset-root "$DATA" \
37             --output-root "$OUT" \
38             "${EXTRA_ARGS[@]}" \
39             > "$LOGDIR/seg_npu${npu}_${start}_${end}.log" 2>&1 &
40         echo "launched npu${npu} [${start}, ${end}] pid=${!}"
41     }
42
43     launch 2 0 3334
44     launch 3 3334 6668
45     launch 4 6668 10002
46     launch 5 10002 13336
47     launch 6 13336 16670
48     launch 7 16670 20000
49
50     echo ""
51     echo "进度文件: $OUT/meta/progress_npu{2..7}.json"
52     echo "实时监控: python /mnt/yyli/scripts/watch_segment_progress.py"
53     echo "或:      bash /mnt/yyli/scripts/run_segment_6npu.sh --watch # 仅监控, 勿重复启动"

```

```

54 if [[ "$WATCH" -eq 1 ]]; then
55     python /mnt/yyli/scripts/watch_segment_progress.py --output-root "$OUT"
56 fi

```

run_segment_npu2.sh ——分割启动脚本

```

1  #!/usr/bin/env bash
2  # SAM3 segmentation on physical NPU 2. Adjust args as needed.
3  set -euo pipefail
4
5  source /usr/local/Ascend/cann-8.5.2/set_env.sh
6  source /home/yyli/miniconda3/etc/profile.d/conda.sh
7  conda activate sam3
8
9  # CANN 算子编译会 import 昇腾自带的 Python 模块, 缺 decorator 等会报 GEInitialize failed
10 python -c "import decorator" 2>/dev/null || pip install -q decorator attrs scipy absl-py
    cloudpickle ml-dtypes tornado psutil
11
12 export HF_HOME="${HF_HOME:-/mnt/yyli/cache/huggingface}"
13
14 python /mnt/yyli/scripts/segment_aloha_sam3.py \
15     --npu-id 2 \
16     --checkpoint /mnt/sam3/sam3/sam3.pt \
17     --dataset-root /mnt/yyli/data/aloha_sim_transfer_cube_human \
18     --output-root /mnt/yyli/data/aloha_sim_transfer_cube_human_seg \
19     "$@"

```

watch_segment_progress.py ——多卡分割进度实时监控

```

1  #!/usr/bin/env python3
2  """Live dashboard for multi-NPU SAM3 segmentation progress."""
3
4  from __future__ import annotations
5
6  import argparse
7  import json
8  import os
9  import sys
10 import time
11 from pathlib import Path
12
13
14 def fmt_eta(sec: float | None) -> str:
15     if sec is None:
16         return "-"
17     sec = int(sec)
18     h, rem = divmod(sec, 3600)
19     m, s = divmod(rem, 60)

```

```

20     if h:
21         return f"{h}h{m:02d}m"
22     if m:
23         return f"{m}m{s:02d}s"
24     return f"{s}s"
25
26
27 def bar(pct: float, width: int = 28) -> str:
28     pct = max(0.0, min(100.0, pct))
29     filled = int(width * pct / 100)
30     return "#" * filled + "-" * (width - filled)
31
32
33 def load_progress(path: Path) -> dict | None:
34     if not path.is_file():
35         return None
36     try:
37         return json.loads(path.read_text(encoding="utf-8"))
38     except (json.JSONDecodeError, OSError):
39         return None
40
41
42 def render(meta_dir: Path, npu_ids: list[int]) -> str:
43     lines = []
44     lines.append(f"SAM3 分割进度 {time.strftime('%Y-%m-%d %H:%M:%S')}")
45     lines.append("=" * 88)
46     lines.append(
47         f"{'NPU':>4} {'状态':<14} {'进度':<32} {'done/total':>12} {'fps':>6} {'ETA':>8} {'idx'
48         ':>8}"
49     )
50     lines.append("-" * 88)
51
52     total_done = 0
53     total_all = 0
54     active = 0
55
56     for npu in npu_ids:
57         p = load_progress(meta_dir / f"progress_npu{npu}.json")
58         if p is None:
59             lines.append(f"{'npu':>4} {'(未启动)':<14} {'':<32} {'-':>12} {'-':>6} {'-':>8}
60             {'-':>8}")
61             continue
62
63         status = p.get("status", "?")
64         done = int(p.get("done", 0))
65         total = int(p.get("total", 0))
66         pct = float(p.get("pct", 0.0))
67         fps = p.get("frames_per_sec")

```

```

66     eta = p.get("eta_sec")
67     gidx = p.get("current_global_index")
68     skipped = int(p.get("skipped", 0))
69     failed = int(p.get("failed", 0))
70
71     total_done += done
72     total_all += total
73     if status not in ("done",):
74         active += 1
75
76     prog = f"{bar(pct)} {pct:5.1f}%"
77     done_str = f"{done}/{total}"
78     fps_str = f"{fps:.2f}" if isinstance(fps, (int, float)) else "-"
79     idx_str = str(gidx) if gidx is not None else "-"
80     extra = ""
81     if skipped or failed:
82         extra = f" skip={skipped} fail={failed}"
83
84     lines.append(
85         f"{npu:>4} {status:<14} {prog:<32} {done_str:>12} {fps_str:>6} {fmt_eta(eta):>8}
{idx_str:>8}{extra}"
86     )
87
88     lines.append("-" * 88)
89     overall_pct = 100.0 * total_done / total_all if total_all else 0.0
90     lines.append(
91         f"合计: {total_done}/{total_all} ({overall_pct:.1f}%) 活跃卡数: {active}/{len(npu_ids)}"
92     )
93     lines.append("Ctrl+C 退出")
94     return "\n".join(lines)
95
96
97 def parse_args() -> argparse.Namespace:
98     parser = argparse.ArgumentParser(description="Watch SAM3 segmentation progress")
99     parser.add_argument(
100         "--output-root",
101         type=Path,
102         default=Path("/mnt/yyli/data/aloha_sim_transfer_cube_human_seg"),
103     )
104     parser.add_argument(
105         "--npu-ids",
106         type=int,
107         nargs="+",
108         default=[2, 3, 4, 5, 6, 7],
109         help="Physical NPU ids to monitor",
110     )
111     parser.add_argument("--interval", type=float, default=2.0, help="Refresh interval (seconds)")
112     return parser.parse_args()

```

```

113
114
115 def main() -> None:
116     args = parse_args()
117     meta_dir = args.output_root.expanduser().resolve() / "meta"
118
119     try:
120         while True:
121             text = render(meta_dir, args.npu_ids)
122             if sys.stdout.isatty():
123                 os.system("clear")
124                 print(text, flush=True)
125                 time.sleep(args.interval)
126         except KeyboardInterrupt:
127             print("\n已停止监控。")
128
129
130 if __name__ == "__main__":
131     main()

```

pairwise.py ——构建成对距离矩阵（输入几何距离与梯度空间距离）

```

1 """
2 构建成对距离矩阵 M[i,j]: 训练池里任意两帧之间的差异，选择算法就拿它当“距离”。
3 两种输入几何距离:
4     mode=1: 动作距离和视觉距离加权相加 M = w_act * minmax(||a_i-a_j||) + w_vis * minmax(1-cos(f_i,f_j))
5     mode=2: 只看动作 M = minmax(||a_i-a_j||)
6 矩阵很大，存成 float16 内存映射（memmap）、分块计算以省内存。距离依赖动作，所以左右臂各建一份。
7
8 算动作距离用的向量不止原始 7 维位置，还做了两点处理：
9     逐维标准化（standardize_act）：每维做 z-score，消掉量纲差异，免得幅度大的关节维主导距离；
10    动力学特征（use_dynamics）：把速度 Δa、加速度 ΔΔa（在 episode 内按时间求差）也拼进来，
11    让差异更能体现“动作变化”而不仅是“姿态不同”。位置/速度/加速度各自 z-score 后按权重拼接。
12 """
13 import os
14 import numpy as np
15
16
17 def _zscore(X, eps):
18     """逐列 z-score（用本训练池自己的均值和方差）。"""
19     return (X - X.mean(0)) / (X.std(0) + eps)
20
21
22 def _dynamics_features(action7, ep_id, frame_id):
23     """在每条 episode 内按时间求差，算速度和加速度（不跨 episode）。返回与输入同序的 (vel, acc)。"""
24     vel = np.zeros_like(action7, dtype=np.float32)
25     acc = np.zeros_like(action7, dtype=np.float32)
26     for e in np.unique(ep_id):

```

```

27     order = np.where(ep_id == e)[0]
28     order = order[np.argsort(frame_id[order])]          # 这条 episode 的时间序
29     seq = action7[order].astype(np.float32)
30     v = np.diff(seq, axis=0, prepend=seq[:1])          # 速度, 首帧记 0
31     a = np.diff(v, axis=0, prepend=v[:1])              # 加速度, 首帧记 0
32     vel[order], acc[order] = v, a
33     return vel, acc
34
35
36 def _build_action_repr(action7, ep_id, frame_id, eps,
37                        standardize_act, use_dynamics, w_pos, w_vel, w_acc):
38     """拼出算动作距离用的表征: [位置 | 速度 | 加速度], 每块各自 z-score 后按权重拼接。
39     两个开关都关掉时就退回原始 7 维动作。"""
40     pos = action7.astype(np.float32)
41     blocks = [w_pos * (_zscore(pos, eps) if standardize_act else pos)]
42     if use_dynamics and ep_id is not None and frame_id is not None:
43         vel, acc = _dynamics_features(pos, ep_id, frame_id)
44         if standardize_act:
45             vel, acc = _zscore(vel, eps), _zscore(acc, eps)
46         blocks += [w_vel * vel, w_acc * acc]
47     return np.concatenate(blocks, axis=1).astype(np.float32)
48
49
50 def build_pairwise_M(img_feat, action7, out_path, mode=1,
51                     w_act=0.7, w_vis=0.3, block=1000, eps=1e-8,
52                     ep_id=None, frame_id=None, standardize_act=True,
53                     use_dynamics=True, w_pos=1.0, w_vel=1.0, w_acc=0.5):
54     """构建并保存 N×N 的 float16 memmap 距离矩阵 M, 返回它。
55     传了 ep_id/frame_id 才能算速度/加速度; 不传就只用位置。"""
56     N = img_feat.shape[0]
57     A = _build_action_repr(action7, ep_id, frame_id, eps,
58                           standardize_act, use_dynamics, w_pos, w_vel, w_acc)
59     A2 = (A * A).sum(1)                                # 每行的平方和, 算欧氏距离用
60     if mode == 1:
61         F = img_feat.astype(np.float32)
62         Fn = F / (np.linalg.norm(F, axis=1, keepdims=True) + eps)  # 单位化, 用来算余弦
63
64     def _blocks():
65         for r0 in range(0, N, block):
66             yield r0, min(r0 + block, N)
67
68     def _d_act(r0, r1):
69         # 这一块行相对全体列的动作欧氏距离
70         d2 = A2[r0:r1, None] + A2[None, :] - 2.0 * (A[r0:r1] @ A.T)
71         np.maximum(d2, 0, out=d2)
72         return np.sqrt(d2)
73
74     # 第一遍: 扫一遍拿到全局 min/max, 供后面 min-max 归一化

```

```

75     amin, amax = np.inf, -np.inf
76     vmin, vmax = np.inf, -np.inf
77     for r0, r1 in _blocks():
78         da = _d_act(r0, r1)
79         amin, amax = min(amin, da.min()), max(amax, da.max())
80         if mode == 1:
81             dv = 1.0 - (Fn[r0:r1] @ Fn.T)
82             vmin, vmax = min(vmin, dv.min()), max(vmax, dv.max())
83
84     # 第二遍: 归一化、加权, 写进 memmap
85     os.makedirs(os.path.dirname(out_path), exist_ok=True)
86     M = np.memmap(out_path, dtype=np.float16, mode="w+", shape=(N, N))
87     for r0, r1 in _blocks():
88         m = (_d_act(r0, r1) - amin) / (amax - amin + eps)
89         if mode == 1:
90             dv = 1.0 - (Fn[r0:r1] @ Fn.T)
91             m = w_act * m + w_vis * (dv - vmin) / (vmax - vmin + eps)
92         M[r0:r1] = m.astype(np.float16)
93     for i in range(N):                                # 对角线 (自身距离) 置 0
94         M[i, i] = 0
95     M.flush()
96     return M
97
98
99 def build_grad_M(G, out_path, block=1000, eps=1e-8):
100     """从 pilot 模型的末层梯度特征  $G(N \times d)$  建距离矩阵  $M = \min_{i,j}(1 - \cos(g_i, g_j))$ 。
101     用于 CRAIG: 在“梯度方向”空间里做 facility 选择和样本加权, 让加权子集的梯度逼近全量梯度。
102     输出格式 (float16 memmap、分块) 和 build_pairwise_M 一致。"""
103     N = G.shape[0]
104     Gn = G.astype(np.float32)
105     Gn = Gn / (np.linalg.norm(Gn, axis=1, keepdims=True) + eps) # 单位化, 用来算余弦
106
107     def _blocks():
108         for r0 in range(0, N, block):
109             yield r0, min(r0 + block, N)
110
111     # 第一遍: 全局 min/max (1-cos 落在 [0,2], 因为梯度方向可能相反)
112     dmin, dmax = np.inf, -np.inf
113     for r0, r1 in _blocks():
114         d = 1.0 - (Gn[r0:r1] @ Gn.T)
115         dmin, dmax = min(dmin, d.min()), max(dmax, d.max())
116
117     os.makedirs(os.path.dirname(out_path), exist_ok=True)
118     M = np.memmap(out_path, dtype=np.float16, mode="w+", shape=(N, N))
119     for r0, r1 in _blocks():
120         d = 1.0 - (Gn[r0:r1] @ Gn.T)
121         M[r0:r1] = ((d - dmin) / (dmax - dmin + eps)).astype(np.float16)
122     for i in range(N):

```

```

123         M[i, i] = 0
124     M.flush()
125     return M
126
127
128 def global_quantile_from_M(M, q):
129     """取 M 上三角 (i<j) 所有元素的 q 分位数。用 partition 取第 k 小, O(n) 且省内存。"""
130     N = M.shape[0]
131     total = N * (N - 1) // 2
132     packed = np.empty(total, dtype=np.float16)
133     pos = 0
134     for i in range(N - 1):
135         row = M[i, i + 1:]
136         packed[pos:pos + row.shape[0]] = row
137         pos += row.shape[0]
138     k = int(q * total)
139     return float(np.partition(packed, k)[k])

```

coreset.py —— 关键帧锚点、facility 代表点选择、CRAIG 权重与随机基准

```

1  """
2  核心集选择: 从训练池里挑出一小撮最有价值的帧。
3  补齐方式由 method 决定:
4      fps      —— 最近点采样, 让选中的点尽量铺开 (k-center)。
5      facility —— 代表点选择, 让选中的点落在稠密区中心、贴合数据分布 (k-median)。
6  下面所有函数都在“训练池内部的局部下标”上工作, 返回的也是局部下标, 由调用方映射回全局帧号。
7  """
8  import heapq
9  import numpy as np
10
11
12 def _minmax(x, eps=1e-8):
13     # 把任意范围线性压到 [0,1], 免得量纲大的分量主导
14     lo, hi = x.min(), x.max()
15     return (x - lo) / (hi - lo + eps)
16
17
18 # ----- 夹物体帧检测 -----
19 # 夹具信号约定 (实测确认): 值高 = 张开, 值低 = 闭合。
20 # 抓取动作是“先张开手靠近物体、再合拢夹住”, 所以真正的抓取帧是一段“张开→闭合”的下降沿,
21 # 而不是夹具张得最大的那一刻。右臂常有两段开合 (先抓后放), 这里只取第一段闭合, 后面的放下忽略。
22 def detect_grasp_frame(g, grasp_thr=0.5, eps=1e-8):
23     """输入某条 episode 按时间排列的夹具开合值, 返回抓取帧在该 episode 内的位置;
24     整段几乎没动就返回 None。"""
25     g = np.asarray(g, dtype=np.float64)
26     n = len(g)
27     if n < 5:
28         return None

```

```

29
30 # 先轻度滑动平均去抖；两端用 edge 填充，免得卷积在边界把值往 0 拉
31 k = max(3, n // 50)
32 if k % 2 == 0:
33     k += 1
34 pad = k // 2
35 gs = np.convolve(np.pad(g, pad, mode="edge"), np.ones(k) / k, mode="valid")
36
37 lo, hi = np.percentile(gs, 5), np.percentile(gs, 95) # 取 5/95 分位当上下界，抗离群
38 span = hi - lo
39 if span < eps:
40     return None # 夹具全程没动
41
42 # 一对带间隔的阈值（滞回），避免在分界线附近反复横跳
43 open_thr = lo + min(grasp_thr + 0.10, 0.95) * span # 高于它算“张开”
44 close_thr = lo + max(grasp_thr - 0.10, 0.05) * span # 低于它算“闭合”
45 min_open = max(2, k) # 张开要持续这么多帧才算数
46 min_hold = max(3, n // 40) # 闭合也要持续，滤掉一闪而过的尖峰
47
48 opened, open_run = False, 0
49 for t in range(n):
50     if gs[t] >= open_thr: # 连续张开计数
51         open_run += 1
52         if open_run >= min_open:
53             opened = True
54     else:
55         open_run = 0
56     if opened and gs[t] <= close_thr: # 张开过之后第一次落到闭合 = 抓取下降沿
57         end = min(n, t + min_hold)
58         if np.all(gs[t:end] <= open_thr): # 确认之后确实保持闭合（真夹住了）
59             win = min(n, t + 2 * min_hold)
60             return t + int(np.argmax(gs[t:win])) # 取这段里夹得最紧的一帧
61 return None
62
63
64 def _episode_groups(ep_id, frame_id):
65     """按 episode 分组、组内按时间排序，返回 {episode: 局部帧下标}。"""
66     groups = {}
67     for e in np.unique(ep_id):
68         idx = np.where(ep_id == e)[0]
69         groups[int(e)] = idx[np.argsort(frame_id[idx])]
70     return groups
71
72
73 def _episode_budgets(lengths, B_total, budget_ratio):
74     """给每条 episode 分名额：大致按长度成比例，用最大余数法凑到总数恰好 B_total。"""
75     raw = [budget_ratio * L for L in lengths]
76     floor = [int(np.floor(r)) for r in raw]

```

```

77     deficit = B_total - sum(floor)
78     order = np.argsort([r - f for r, f in zip(raw, floor)])::-1]
79     for i in range(max(deficit, 0)):
80         floor[order[i % len(order)]] += 1
81     return floor
82
83
84 def _fps(M, seeds, cand_idx, k):
85     """最远点采样: 从 seeds 出发, 每次挑离已选集合最远的点, 直到总数到 k。距离用 M, 返回选中下标。"""
86     cand = np.asarray(cand_idx, dtype=np.int64)
87     pos = {int(c): i for i, c in enumerate(cand)}
88     chosen = list(dict.fromkeys(int(s) for s in seeds))
89     min_d = np.full(len(cand), np.inf, dtype=np.float64)
90     for s in chosen:                                # 每个候选到已选集合的最近距离
91         min_d = np.minimum(min_d, M[s][cand].astype(np.float64))
92     for c in chosen:                                # 已选中的点不再参与挑选
93         if c in pos:
94             min_d[pos[c]] = -np.inf
95     while len(chosen) < k:
96         j = int(np.argmax(min_d))
97         if min_d[j] == -np.inf:
98             break
99         x = int(cand[j])
100        chosen.append(x)
101        min_d[j] = -np.inf
102        min_d = np.minimum(min_d, M[x][cand].astype(np.float64))    # 选完一个就更新最近距离
103    return chosen
104
105
106 def _trim(M, selected, anchor_set, target):
107     """选多了往回删: 反复去掉“离其它已选点最近”的那个非锚点, 直到只剩 target 个。"""
108     sel = list(selected)
109     while len(sel) > target:
110         arr = np.array(sel, dtype=np.int64)
111         removable = [s for s in sel if s not in anchor_set]
112         if not removable:
113             break
114         worst, worst_d = removable[0], np.inf
115         for s in removable:
116             row = M[s][arr].astype(np.float64)
117             row = row[row > 0]                                # 去掉自身 (距离 0)
118             d = float(row.min()) if row.size else 0.0
119             if d < worst_d:
120                 worst_d, worst = d, s
121         sel.remove(worst)
122     return set(sel)
123
124

```

```

125 # ----- 锚点: 首帧/末帧/夹物体帧 (fps 和 facility 都用) -----
126 def _collect_anchors(groups, action7, grasp_thr, eps):
127     """逐 episode 取首帧、末帧、夹物体帧。
128     返回三类锚点各自的 {episode: 局部下标}, 外加 {episode: [首, 末, (夹物体)]} (保持顺序)。"""
129     first_a, last_a, grasp_a, anchor_map = {}, {}, {}, {}
130     for e in sorted(groups.keys()):
131         idx = groups[e]
132         first, last = int(idx[0]), int(idx[-1])
133         first_a[e], last_a[e] = first, last
134         anchors = [first, last]
135         gp = detect_grasp_frame(action7[idx][:, 6], grasp_thr, eps) # 第 7 维是夹具
136         if gp is not None:
137             grasp = int(idx[gp])
138             grasp_a[e] = grasp
139             anchors.append(grasp)
140         anchor_map[e] = anchors
141     return first_a, last_a, grasp_a, anchor_map
142
143
144 def _dedup_anchors(M, first_a, last_a, grasp_a, thr):
145     """同一类锚点跨 episode 去重: 两条 episode 的同类锚点若太近 (距离 < thr), 只留下标小的那个。
146     返回需要删除的下标集合。"""
147     deleted = set()
148     for amap in (first_a, last_a, grasp_a):
149         kept = []
150         for e in sorted(amap.keys()):
151             a = amap[e]
152             if any(float(M[a, b]) < thr for b in kept):
153                 deleted.add(a)
154             else:
155                 kept.append(a)
156     return deleted
157
158
159 # ----- facility-location 代表点选择 -----
160 def _facility_select(M, preselected, cand_idx, k, sigma, eps=1e-8, weight=None):
161     """从 preselected 出发, 在 cand_idx 里贪心挑代表点直到总数到 k。
162     相似度  $s(i,j)=\exp(-(M[i,j]/\sigma)^2)$ , 越近越大。
163     每轮挑 “能让全体点总覆盖增加最多” 的候选: 增益 =  $\sum_i u_i \cdot \max(0, s(i,c)-cover[i])$ 。
164     weight=u 是每个点的重要度 (高信息帧权重重大, 该区域就多分代表点); 不传则全为 1, 即普通 facility。
165     覆盖函数是子模的, 所以用 CELF 懒惰评估加速, 结果和朴素贪心一致但省去大量重复计算。"""
166     N = M.shape[0]
167     if sigma is None or sigma <= 0: # 没给带宽就抽样取距离中位数兜底
168         rng = np.random.default_rng(0)
169         s = M[rng.choice(N, size=min(N, 1000), replace=False)].astype(np.float64)
170         s = s[s > 0]
171         sigma = float(np.median(s)) if s.size else 1.0
172

```

```

173     u = 1.0 if weight is None else np.asarray(weight, dtype=np.float64)    # 覆盖的重要度权重
174
175     def sim_row(j):
176         return np.exp(-(M[j].astype(np.float64) / sigma) ** 2)
177
178     chosen = list(dict.fromkeys(int(p) for p in preselected))
179     cset = set(chosen)
180     cover = np.zeros(N, dtype=np.float64)    # cover[i]: 已选集合对点 i 的最大相似度
181     for p in chosen:
182         np.maximum(cover, sim_row(p), out=cover)
183
184     def gain(c):
185         return float((u * np.maximum(sim_row(c) - cover, 0.0)).sum())
186
187     # CELF 堆元素: (-当前增益, 候选, 这个增益是在已选多少个时算出来的)
188     heap = [(-gain(int(c)), int(c), len(chosen)) for c in cand_idx if int(c) not in cset]
189     heapq.heapify(heap)
190
191     while len(chosen) < k and heap:
192         neg, c, when = heapq.heappop(heap)
193         if when == len(chosen):
194             chosen.append(c)
195             cset.add(c)
196             np.maximum(cover, sim_row(c), out=cover)
197         else:
198             # 增益过期了, 用最新 cover 重算再放回堆
199             heapq.heappush(heap, (-gain(c), c, len(chosen)))
200     return chosen
201
202 # ----- 主流程: 锚点 + 去重 + 补齐 -----
203 def pcras_select(M, action7, ep_id, frame_id,
204                 budget_ratio=0.10, grasp_thr=0.5, thr=0.0, eps=1e-8,
205                 method="fps", sigma=None, importance=None):
206     """从训练池选出核心集, 返回选中的局部下标 (长度 = round(budget_ratio*N)) 和抓取帧字典。
207     参数:
208         M          训练池内的成对距离矩阵 (N×N, 也兼作选择用的距离)
209         thr        锚点去重阈值 (一般取距离的某个分位数)
210         method     补齐方式: "fps" (最远点采样) 或 "facility" (代表点选择)
211         sigma      facility 的相似度带宽; 不传则内部按中位数兜底
212         importance 每帧重要度 (仅 facility 用, 长度 N); 不传则等权
213     """
214     N = M.shape[0]
215     B_total = round(budget_ratio * N)
216     groups = _episode_groups(ep_id, frame_id)
217     eps_sorted = sorted(groups.keys())
218
219     # 三类锚点: 两种补齐方式都先把它们放进来
220     first_a, last_a, grasp_a, anchor_map = _collect_anchors(groups, action7, grasp_thr, eps)

```

```

221
222     if method == "facility":
223         # 第一步: 只放锚点, 剩下的名额全留给第三步的代表点补齐
224         selected = set()
225         for e in eps_sorted:
226             selected.update(anchor_map[e])
227         anchors_all = set(selected)
228
229         # 第二步: 锚点跨 episode 去重, 去掉彼此太相似的
230         deleted = _dedup_anchors(M, first_a, last_a, grasp_a, thr)
231         selected -= deleted
232         anchors_all -= deleted
233
234         # 第三步: 用 facility 补齐到 B_total (极少数情况选多了就裁掉)
235         if len(selected) < B_total:
236             cand = [i for i in range(N) if i not in selected]
237             selected = set(_facility_select(M, list(selected), cand, B_total, sigma, eps,
238                                           weight=importance))
239         elif len(selected) > B_total:
240             selected = _trim(M, selected, anchors_all, B_total)
241
242         return np.array(sorted(selected), dtype=np.int64), grasp_a
243
244     # ===== FPS 路径 =====
245     budgets = _episode_budgets([len(groups[e]) for e in eps_sorted], B_total, budget_ratio)
246     selected, anchors_all = set(), set()
247
248     # 第一步: 在每条 episode 内, 以三个锚点为种子做 FPS, 填满该 episode 的名额
249     for e, be in zip(eps_sorted, budgets):
250         idx = groups[e]
251         anchors = anchor_map[e]
252         be = max(be, len(set(anchors)))
253         selected.update(_fps(M, anchors, idx.tolist(), be))
254         anchors_all.update(anchors)
255
256     # 第二步: 锚点跨 episode 去重
257     deleted = _dedup_anchors(M, first_a, last_a, grasp_a, thr)
258     selected -= deleted
259     anchors_all -= deleted
260
261     # 第三步: 全局 FPS 补齐到 B_total (选多了就裁掉)
262     if len(selected) < B_total:
263         selected = set(_fps(M, list(selected), list(range(N)), B_total))
264     elif len(selected) > B_total:
265         selected = _trim(M, selected, anchors_all, B_total)
266
267     return np.array(sorted(selected), dtype=np.int64), grasp_a
268

```

```

269
270 # ----- 每帧重要度 (给 facility 当覆盖权重用) -----
271 def frame_importance(action7, ep_id, frame_id, eps=1e-8):
272     """每帧重要度 = 动作速度大小 + 加速度大小 (在 episode 内按时间求差, 不跨 episode),
273     最后整体归一化到均值 1。动作越剧烈 (抓取、交接等) 权重越大。返回与输入同序的非负向量。"""
274     a = action7.astype(np.float64)
275     imp = np.zeros(len(a), dtype=np.float64)
276     for e in np.unique(ep_id):
277         order = np.where(ep_id == e)[0]
278         order = order[np.argsort(frame_id[order])] # 这条 episode 的时间序
279         seq = a[order]
280         vel = np.diff(seq, axis=0, prepend=seq[:1]) # 速度, 首帧记 0
281         acc = np.diff(vel, axis=0, prepend=vel[:1]) # 加速度, 首帧记 0
282         imp[order] = np.linalg.norm(vel, axis=1) + np.linalg.norm(acc, axis=1)
283     return imp / (imp.mean() + eps) # 归一到均值 1 (权重是相对的)
284
285
286 # ----- CRAIG 样本权重 -----
287 def craig_weights(M, selected, sigma, eps=1e-8):
288     """给每个选中点算一个权重 = 全体点里“离它最近”的样本数 (它代表的那一簇有多大)。
289     相似度同样用  $s(i,j)=\exp(-(M[i,j]/\sigma)^2)$ 。这就是 CRAIG 的权重: 加权之后子集的梯度能
290     近似全量的梯度, 从而纠正等权选择带来的系统偏差。
291     selected 要和训练时的样本顺序一致; 返回同序、均值归一到 1 的权重。"""
292     sel = np.asarray(selected, dtype=np.int64)
293     N = M.shape[0]
294     if sigma is None or sigma <= 0:
295         rng = np.random.default_rng(0)
296         s = M[rng.choice(N, size=min(N, 1000), replace=False)].astype(np.float64)
297         s = s[s > 0]
298         sigma = float(np.median(s)) if s.size else 1.0
299
300     best_sim = np.full(N, -np.inf, dtype=np.float64) # 每个点到已选集合的最大相似度
301     best_pos = np.zeros(N, dtype=np.int64) # 取到这个最大值的是哪个选中点
302     for pos, j in enumerate(sel):
303         sim_j = np.exp(-(M[int(j)].astype(np.float64) / sigma) ** 2)
304         better = sim_j > best_sim
305         best_sim[better] = sim_j[better]
306         best_pos[better] = pos
307     counts = np.bincount(best_pos, minlength=len(sel)).astype(np.float64)
308     counts = np.maximum(counts, eps) # 每个代表点至少代表自己, 避免 0 权重
309     return counts * (len(sel) / counts.sum()) # 归一到均值 1, 不改变损失整体尺度
310
311
312 # ----- 随机基准 -----
313 def random_select(n_pool, B, seed=0):
314     """从 [0, n_pool] 里随机取 B 个不重复下标。"""
315     return np.random.default_rng(seed).choice(n_pool, size=B, replace=False)

```

mlp.py ——轻量级 MLP 的训练/评估，以及 CRAIG 用的末层梯度特征

```
1  """轻量级 MLP: 把 [视觉特征 + 语言特征] 回归到 7 自由度动作; 含训练/评估, 以及给 CRAIG 用的梯度特征。
   """
2  import numpy as np
3  import torch
4  import torch.nn as nn
5
6
7  class MLP(nn.Module):
8      def __init__(self, in_dim, out_dim=7, hidden=(256, 128), p=0.1):
9          super().__init__()
10         layers, d = [], in_dim
11         for h in hidden:
12             layers += [nn.Linear(d, h), nn.ReLU(), nn.Dropout(p)]
13             d = h
14         layers += [nn.Linear(d, out_dim)]
15         self.net = nn.Sequential(*layers)
16
17     def forward(self, x):
18         return self.net(x)
19
20
21 def _standardize_fit(X, eps=1e-8):
22     # 返回均值和标准差用于输入标准化; 文本那几列是常量, std eps, 标准化后 0, 不会出问题
23     return X.mean(0), X.std(0) + eps
24
25
26 def train_eval(Xtr, Ytr, Xte, Yte, cfg, seed=0, device=None, Yte_orig=None, y_mu=None, y_std=None,
27               sample_weight=None):
28     """在选好的训练集上训练 MLP, 在测试集上算 MSE。
29     Xtr/Xte 是输入特征, Ytr/Yte 是 7 自由度动作目标 (原始或归一化)。
30     sample_weight 是每个训练样本的权重 (长度 = len(Xtr), CRAIG 用); 不传则全部等权, 和普通训练一样。
31     返回 (整体 MSE, 逐关节 MSE[7])。"""
32     device = device or ("cuda" if torch.cuda.is_available() else "cpu")
33     torch.manual_seed(seed); np.random.seed(seed)
34
35     mu, sd = _standardize_fit(Xtr)          # 输入标准化 (用训练集统计)
36     Xtr_ = (Xtr - mu) / sd
37     Xte_ = (Xte - mu) / sd
38
39     model = MLP(Xtr.shape[1], Ytr.shape[1], cfg.HIDDEN, cfg.DROPOUT).to(device)
40     opt = torch.optim.Adam(model.parameters(), lr=cfg.LR, weight_decay=cfg.WEIGHT_DECAY)
41
42     # 加权 MSE: 先算每个样本的 MSE, 再按权重求加权平均; 权重全 1 时就等于普通 MSELoss
43     if sample_weight is None:
44         sample_weight = np.ones(len(Xtr), dtype=np.float32)
45     W = torch.tensor(np.asarray(sample_weight, dtype=np.float32))
```

```

46
47     dl = torch.utils.data.DataLoader(
48         torch.utils.data.TensorDataset(
49             torch.tensor(Xtr_, dtype=torch.float32),
50             torch.tensor(Ytr, dtype=torch.float32),
51             W),
52         batch_size=cfg.BATCH_SIZE, shuffle=True)
53
54     print(f" [Train] Size: {Xtr.shape[0]} | Epochs: {cfg.EPOCHS} | Device: {device}")
55     model.train()
56     for epoch in range(cfg.EPOCHS):
57         epoch_loss = 0.0
58         for xb, yb, wb in dl:
59             xb, yb, wb = xb.to(device), yb.to(device), wb.to(device)
60             opt.zero_grad()
61             se = ((model(xb) - yb) ** 2).mean(dim=1)          # 每个样本对 7 维取均值
62             loss = (wb * se).sum() / wb.sum()                # 加权平均
63             loss.backward()
64             opt.step()
65             epoch_loss += loss.item() * xb.size(0)
66
67             if (epoch + 1) % 40 == 0 or epoch == 0:          # 每 40 轮打一次进度
68                 print(f"    Epoch {epoch+1:03d}/{cfg.EPOCHS} | Loss: {epoch_loss / len(dl.dataset):.6f
69 }")
70
71     model.eval()
72     with torch.no_grad():
73         pred = model(torch.tensor(Xte_, dtype=torch.float32).to(device)).cpu().numpy()
74
75     if getattr(cfg, "EVAL_ORIGINAL", False):
76         if getattr(cfg, "SUPERVISE_ORIGINAL", False):
77             # 训练目标就是原始动作, pred 也是原始动作, 直接和 Yte (原始动作) 比
78             mse = float(np.mean((pred - Yte) ** 2))
79             per_joint = np.mean((pred - Yte) ** 2, axis=0)
80         else:
81             # 训练目标是归一化动作, 要先反归一化再和原始动作 Yte_orig 比
82             pred_orig = pred * y_std + y_mu
83             mse = float(np.mean((pred_orig - Yte_orig) ** 2))
84             per_joint = np.mean((pred_orig - Yte_orig) ** 2, axis=0)
85     else:
86         # 直接在归一化空间里算 MSE
87         mse = float(np.mean((pred - Yte) ** 2))
88         per_joint = np.mean((pred - Yte) ** 2, axis=0)
89
90     return mse, per_joint
91
92 def pilot_gradient_features(Xtr, Ytr, cfg, seed=0, device=None, block=2048):

```

```

93     """训练一个 pilot MLP（设置和 train_eval 一样），返回每个训练样本的末层梯度特征 G(N×d)。
94     对每帧取：残差 err = 预测 - 真值，以及末层输入 h（隐层激活），拼成 g = [flatten(err h), err]，
95     它就是这一帧损失对末层参数的梯度，刻画“这一帧想把模型往哪个方向更新”。
96     CRAIG 就是在这个梯度空间里选代表点、配权重，让加权子集的梯度逼近全量梯度。"""
97     device = device or ("cuda" if torch.cuda.is_available() else "cpu")
98     torch.manual_seed(seed); np.random.seed(seed)
99     mu, sd = _standardize_fit(Xtr)
100     Xtr_ = ((Xtr - mu) / sd).astype(np.float32)
101     Y = np.asarray(Ytr, dtype=np.float32)
102
103     model = MLP(Xtr.shape[1], Y.shape[1], cfg.HIDDEN, cfg.DROPOUT).to(device)
104     opt = torch.optim.Adam(model.parameters(), lr=cfg.LR, weight_decay=cfg.WEIGHT_DECAY)
105     lossf = nn.MSELoss()
106     dl = torch.utils.data.DataLoader(
107         torch.utils.data.TensorDataset(torch.tensor(Xtr_), torch.tensor(Y)),
108         batch_size=cfg.BATCH_SIZE, shuffle=True)
109
110     print(f" [Pilot] Size: {Xtr.shape[0]} | Epochs: {cfg.EPOCHS} | Device: {device}")
111     model.train()
112     for epoch in range(cfg.EPOCHS):
113         for xb, yb in dl:
114             xb, yb = xb.to(device), yb.to(device)
115             opt.zero_grad(); lossf(model(xb), yb).backward(); opt.step()
116
117     # 训练完做一次前向，分块取末层梯度特征（eval 模式下 Dropout 关闭）
118     model.eval()
119     feats = []
120     with torch.no_grad():
121         for r0 in range(0, len(Xtr_), block):
122             r1 = min(r0 + block, len(Xtr_))
123             x = torch.tensor(Xtr_[r0:r1]).to(device)
124             h = model.net[:-1](x) # 末层之前的隐层输出
125             pred = model.net[-1](h)
126             err = pred - torch.tensor(Y[r0:r1]).to(device) # 残差
127             outer = torch.einsum('bi,bj->bij', err, h).reshape(r1 - r0, -1)
128             g = torch.cat([outer, err], dim=1) # [err h | err]
129             feats.append(g.cpu().numpy().astype(np.float32))
130     return np.concatenate(feats, axis=0)

```

run_experiment.py ——全量 / 随机 / 核心集多种子对比主程序

```

1     """
2     主实验入口：在同一测试集上比较 全量(100%) / 随机(10%) / 核心集(10%) 三类方法的 MSE。
3     默认左右臂各跑一遍再取平均（config.ARM="both"）。需先跑 extract_features.py 生成特征文件。
4     用法：python run_experiment.py [task_id]
5     """
6     import os
7     import json

```

```

8 import argparse
9 import numpy as np
10 import config as cfg
11 from coreset import pcras_select, random_select, frame_importance, craig_weights
12 from pairwise import build_pairwise_M, build_grad_M, global_quantile_from_M
13 from mlp import train_eval, pilot_gradient_features
14
15
16 def run_for_arm(arm, img, txt, action, ep_id, frame_id, tr, te, task_id, seg_feats=None):
17     """对单只臂跑全量/随机/核心集三类方法，返回 {方法名: (MSE 均值, MSE 标准差)}。
18
19     img      : 整图 CLIP 视觉特征 phi，始终用于 MLP 的训练/测试输入。
20     seg_feats : {"left":phi_seg_left, "right":phi_seg_right} 或 None。当 VIS_FEATURE_SRC=="seg"
21                 时，距离矩阵 M 的视觉项改用对应臂的抠图特征 phi_seg；MLP 输入不受影响。
22
23     """
24     N = len(img)
25     action7 = action[:, 7:] if arm == "right" else action[:, :7]
26
27     # MLP 的输入 = 整图图像特征(512) 和文本特征(512) 拼接 —— 任何对照都用整图，不换成抠图
28     F_mlp = np.concatenate([img, np.tile(txt, (N, 1))], axis=1)
29
30     # 距离矩阵 M 的视觉项用哪份特征：整图 phi 还是该臂的抠图 phi_seg
31     if cfg.VIS_FEATURE_SRC == "seg":
32         assert seg_feats is not None, "VIS_FEATURE_SRC=seg 需先跑 extract_seg_features.py 生成
33             seg_features.npz"
34         img_vis = seg_feats[arm]
35     else:
36         img_vis = img
37
38     # 准备归一化后的动作目标：训练集、测试集各用自己的均值/方差归一化
39     Y_all = np.zeros_like(action7)
40     a_mu_tr = action7[tr].mean(0)
41     a_sd_tr = action7[tr].std(0) + cfg.EPS
42     Y_all[tr] = (action7[tr] - a_mu_tr) / a_sd_tr
43     a_mu_te = action7[te].mean(0)
44     a_sd_te = action7[te].std(0) + cfg.EPS
45     Y_all[te] = (action7[te] - a_mu_te) / a_sd_te
46
47     Xte, Yte = F_mlp[te], Y_all[te]
48     B = round(cfg.BUDGET_RATIO * len(tr))
49     res = {}
50
51     # 监督目标用原始动作还是归一化动作
52     eval_kwargs = {}
53     if getattr(cfg, "SUPERVISE_ORIGINAL", False):
54         # 直接用原始动作监督，模型输出就是原始动作
55         Y_tr_target = action7
56         Yte_target = action7[te]

```

```

55     else:
56         # 用归一化动作监督
57         Y_tr_target = Y_all
58         Yte_target = Yte
59         # 若要在原始动作空间评估, 则把反归一化所需的均值/方差传给 train_eval
60         if getattr(cfg, "EVAL_ORIGINAL", False):
61             eval_kwargs = {
62                 "Yte_orig": action7[te],
63                 "y_mu": a_mu_te,
64                 "y_std": a_sd_te
65             }
66
67         # 全量/随机基线: 只跟监督方式有关, 和距离度量/选择方法无关。
68         # 批量扫描时可设 RUN_BASELINES=False 跳过, 免得每个配置都重算一遍。
69         if getattr(cfg, "RUN_BASELINES", True):
70             # 全量 100% (可单独用 RUN_FULL=False 跳过, 只留随机)
71             if getattr(cfg, "RUN_FULL", True):
72                 m, _ = train_eval(F_mlp[tr], Y_tr_target[tr], Xte, Yte_target, cfg, seed=0, **
73                                 eval_kwargs)
74                 res["full (100%)"] = (m, 0.0)
75
76             # 随机 10% (多种子取 mean±std)
77             rand = []
78             for s in cfg.RANDOM_SEEDS:
79                 g = tr[random_select(len(tr), B, seed=s)]
80                 mm, _ = train_eval(F_mlp[g], Y_tr_target[g], Xte, Yte_target, cfg, seed=s, **
81                                 eval_kwargs)
82                 rand.append(mm)
83                 res["random (10%)"] = (float(np.mean(rand)), float(np.std(rand)))
84
85         # 核心集 10%: 先建距离矩阵, 再选点 (视觉项用 img_vis: 整图 phi 或抠图 phi_seg)
86         img_tr, act_tr = img_vis[tr], action7[tr]
87         ep_tr, fr_tr = ep_id[tr], frame_id[tr]
88         arm_m_path = cfg.M_PATH.replace(".dat", f"_{cfg.VIS_FEATURE_SRC}_{arm}.dat")
89         is_craig = (cfg.CORESET_METHOD == "craig") # CRAIG: 在梯度空间里选
90         if is_craig:
91             # 训练 pilot 模型拿到每帧的末层梯度特征, 在“梯度方向”空间建距离矩阵
92             G = pilot_gradient_features(F_mlp[tr], Y_tr_target[tr], cfg, seed=0)
93             M = build_grad_M(G, arm_m_path.replace(".dat", "_grad.dat"), cfg.M_BLOCK, cfg.EPS)
94             sel_method = "facility" # 梯度空间里同样用 facility 选代表点
95         else:
96             # 输入几何距离 (动作, 或动作+视觉)
97             M = build_pairwise_M(img_tr, act_tr, arm_m_path, cfg.SURPRISE_MODE,
98                                 cfg.W_ACT, cfg.W_VIS, cfg.M_BLOCK, cfg.EPS,
99                                 ep_id=ep_tr, frame_id=fr_tr,
100                                 standardize_act=cfg.STANDARDIZE_ACT,
101                                 use_dynamics=cfg.USE_DYNAMICS,
102                                 w_pos=cfg.W_POS, w_vel=cfg.W_VEL, w_acc=cfg.W_ACC)

```

```

101     sel_method = cfg.CORESET_METHOD
102     thr = global_quantile_from_M(M, cfg.Q_PERCENTILE) # 锚点去重阈值
103     # facility、CRAIG、重加权、自适应预算都要用到相似度带宽 sigma
104     need_sigma = (sel_method == "facility"
105                  or getattr(cfg, "REWEIGHT", False)
106                  or getattr(cfg, "ADAPTIVE_BUDGET", False))
107     sigma = global_quantile_from_M(M, cfg.SIGMA_Q) if need_sigma else None
108     # 自适应预算：每帧动力学重要度（仅 facility 用，不开则等权）
109     importance = (frame_importance(act_tr, ep_tr, fr_tr, cfg.EPS)
110                  if getattr(cfg, "ADAPTIVE_BUDGET", False) else None)
111     sel, grasp_a = pcras_select(M, act_tr, ep_tr, fr_tr,
112                                cfg.BUDGET_RATIO, cfg.GRASP_THR, thr, cfg.EPS,
113                                method=sel_method, sigma=sigma, importance=importance)
114
115     # 保存抓取帧索引（存到 OUTPUT_DIR/<task_id>/, 供可视化用）
116     global_grasp = {int(e): int(tr[idx_local]) for e, idx_local in grasp_a.items()}
117     out_dir = os.path.join(cfg.OUTPUT_DIR, task_id)
118     os.makedirs(out_dir, exist_ok=True)
119     grasp_save_path = os.path.join(out_dir, f"grasp_indices_{task_id}_{arm}.json")
120     with open(grasp_save_path, "w", encoding="utf-8") as f:
121         json.dump(global_grasp, f, indent=4)
122     print(f"[{arm}] Saved {len(global_grasp)} grasp frame indices to {grasp_save_path}")
123
124     g = tr[sel]
125     # CRAIG 重加权：给选中点配权重（代表簇大小）做加权训练；craig 方法必须带权重（这是它的核心）
126     use_rw = getattr(cfg, "REWEIGHT", False) or is_craig
127     sw = craig_weights(M, sel, sigma, cfg.EPS) if use_rw else None
128     # 核心集是确定的，只变训练随机种子，跑多种子报 mean±std（口径和随机基准一致，可比）
129     pcr = []
130     for s in cfg.RANDOM_SEEDS:
131         mm, _ = train_eval(F_mlp[g], Y_tr_target[g], Xte, Yte_target, cfg, seed=s,
132                           sample_weight=sw, **eval_kwargs)
133         pcr.append(mm)
134     res["PC-RAS (10%)"] = (float(np.mean(pcr)), float(np.std(pcr)))
135     return res
136
137
138 def _print_table(title, res):
139     space_str = "原始动作空间" if getattr(cfg, "EVAL_ORIGINAL", False) else "归一化动作空间"
140     print(f"\n===== {title} ({space_str} MSE, 越低越好) =====")
141     for k, (m, sd) in res.items():
142         extra = f" ± {sd:.5f}" if sd > 0 else ""
143         print(f"    {k:<16s}: {m:.5f}{extra}")
144     if "random (10%)" in res and "PC-RAS (10%)" in res:
145         r = res["random (10%)"][0]
146         p = res["PC-RAS (10%)"][0]
147         print(f"    PC-RAS vs 随机: 相对降低 {100*(r-p)/r:.1f}% ({'更好' if p < r else '未占优'})")
148

```

```

149
150 def _ratio_entry(per_arm, method, denom, arms):
151     """单个方法的两臂比值平均：每臂先除以该臂随机基线，再对两臂取平均。
152     返回与 aggregate_ratio.py 一致的结构(left_ratio/right_ratio/avg_ratio/rel_pct)。"""
153     ratios = {a: per_arm[a][method][0] / denom[a] for a in arms}
154     avg = float(np.mean(list(ratios.values())))
155     e = {f"{a}_ratio": ratios[a] for a in arms}
156     e["avg_ratio"] = avg
157     e["rel_pct"] = (avg - 1.0) * 100.0
158     e["baseline_random"] = {a: denom[a] for a in arms}
159     return e
160
161
162 def _print_ratio_table(title, res):
163     print(f"\n===== {title} (相对随机比值, <1 更好) =====")
164     for k, e in res.items():
165         print(f"    {k:<16s}: {e['avg_ratio']:.3f}    (相对随机 {e['rel_pct']:+.1f}%)")
166
167
168 def main():
169     parser = argparse.ArgumentParser(description="Run coresnet experiment.")
170     parser.add_argument("task_id", type=str, nargs="?", default=cfg.REPO_ID.split('/')[1],
171                         help="Task ID, 用作输出子目录名 (如 transfer_cube)")
172     args, _ = parser.parse_known_args()
173     task_id = args.task_id
174
175     d = np.load(cfg.FEATURE_PATH)
176     img      = d["img_feat"].astype(np.float32)
177     txt      = d["txt_feat"].astype(np.float32)
178     action   = d["action"].astype(np.float32)
179     ep_id     = d["ep_id"].astype(int)
180     frame_id = d["frame_id"].astype(int)
181
182     # 抠图特征 phi_seg(左右各一份)，仅当用 seg 视觉项时加载并核对外对齐
183     seg_feats = None
184     if cfg.VIS_FEATURE_SRC == "seg":
185         ds = np.load(cfg.SEG_FEATURE_PATH)
186         seg_feats = {"left": ds["img_feat_left"].astype(np.float32),
187                     "right": ds["img_feat_right"].astype(np.float32)}
188         assert len(seg_feats["left"]) == len(img) == len(seg_feats["right"]), "seg 特征与整图特征行
            数不一致"
189         if not (np.array_equal(ds["ep_id"].astype(int), ep_id)
190                and np.array_equal(ds["frame_id"].astype(int), frame_id)):
191             raise ValueError("seg_features 与 clip_features 的 ep_id/frame_id 不对齐, 提取顺序有误")
192     print(f"[vis] 距离矩阵视觉项来源 = {cfg.VIS_FEATURE_SRC} | W_ACT={cfg.W_ACT} W_VIS={cfg.W_VIS
193           }")
194
195     # 按 episode 划分 train/test (45/5)，左右臂共用同一划分

```

```

195     eps_all = np.unique(ep_id)
196     rng = np.random.default_rng(cfg.SEED)
197     test_eps = set(rng.choice(eps_all, size=cfg.N_TEST_EP, replace=False).tolist())
198     test_mask = np.isin(ep_id, list(test_eps))
199     tr = np.where(~test_mask)[0]
200     te = np.where(test_mask)[0]
201     print(f"[split] train {len(tr)} frames / test {len(te)} frames | "
202           f"B={round(cfg.BUDGET_RATIO*len(tr))}")
203
204     # 决定跑哪几只臂
205     if cfg.ARM == "both":
206         arms = ["left", "right"]
207     elif cfg.ARM == "auto":
208         arms = ["right" if action[:, 7:].var() >= action[:, :7].var() else "left"]
209     else:
210         arms = [cfg.ARM]
211
212     per_arm = {}
213     for arm in arms:
214         per_arm[arm] = run_for_arm(arm, img, txt, action, ep_id, frame_id, tr, te, task_id,
215                                   seg_feats)
216         _print_table(f"臂 = {arm}", per_arm[arm])
217
218     # 两只臂取平均: 原始 MSE 算术平均(保留, 旧口径); 相对随机的比值平均(新主口径, 销量纲)
219     avg = {}
220     avg_ratio = {}
221     if len(arms) > 1:
222         # 每臂的随机基线: 本次跑出 random 就用本次的, 否则回退到 config.REF_RANDOM
223         denom = {a: (per_arm[a]["random (10%)"][0] if "random (10%)" in per_arm[a]
224                      else cfg.REF_RANDOM[a]) for a in arms}
225         for method in per_arm[arms[0]]:
226             ms = [per_arm[a][method][0] for a in arms]
227             avg[method] = (float(np.mean(ms)), float(np.std(ms)))
228             avg_ratio[method] = _ratio_entry(per_arm, method, denom, arms)
229         _print_table("左右臂平均 (原始 MSE)", avg)
230         _print_ratio_table("左右臂平均", avg_ratio)
231
232     # 结果写到 OUTPUT_DIR/<task_id>/experiment_results.json
233     results_to_save = {
234         "per_arm": per_arm,
235         "average": avg if len(arms) > 1 else None,
236         "average_relative_to_random": avg_ratio if len(arms) > 1 else None
237     }
238
239     res_dir = os.path.join(cfg.OUTPUT_DIR, task_id)
240     os.makedirs(res_dir, exist_ok=True)
241     res_save_path = os.path.join(res_dir, "experiment_results.json")
242     with open(res_save_path, "w", encoding="utf-8") as f:
243         json.dump(results_to_save, f, indent=4)

```

```

242     print(f"\n[saved] Experiment results saved to {res_save_path}")
243
244
245 if __name__ == "__main__":
246     main()

```

aggregate_ratio.py ——将各实验两臂平均改算为相对随机的比值口径

```

1  """
2  把各实验的"两臂平均"从【原始 MSE 算术平均】改算为【相对随机的比值平均】：
3      每臂先算 ratio_arm = PC-RAS_arm / random_arm，再对左右臂取平均。
4  这样消掉左右臂量级差(右臂 MSE ~ 左臂的 3-4 倍)，两臂等权，直接反映"相对随机好多少"。
5
6  非破坏式：保留各 JSON 原有的 "average"(原始 MSE 平均)，仅新增字段
7      "average_relative_to_random": {"PC-RAS (10%)": {left_ratio,right_ratio,avg_ratio,rel_pct,...}}
8  并在终端打印统一口径对照表。
9
10 随机基线全实验共享(划分 SEED=42 + 训练种子[0..4]一致)，故用同一条参照。
11 运行：python aggregate_ratio.py
12  """
13  import json
14  import os
15
16  OUT = r"E:\vision-roc\expiement-output"
17
18  # 共享随机基线(每臂 random 的 5 种子均值)，取自带基线的 JSON，全实验一致
19  RAND = {"left": 0.0039012286346405745, "right": 0.012428723834455013}
20
21  # 展示标签 -> 结果目录(None 表示随机自身，比值恒为 1)
22  EXPS = [
23      ("随机(10%)",          None),
24      ("度量一·仅动作",      "fac_orig_mode2_multiseed_20260617_1341"),
25      ("度量二·整图",        "m2_full"),
26      ("度量二·抠图",        "m2_seg"),
27      ("纯视觉·整图",        "vis_full"),
28      ("纯视觉·抠图",        "vis_seg"),
29      ("度量三·CRAIG",       "craig_orig_mode1_20260616_2351"),
30  ]
31
32  KEY = "PC-RAS (10%)"
33
34
35  def process(d):
36      p = os.path.join(OUT, d, "experiment_results.json")
37      data = json.load(open(p, encoding="utf-8"))
38      pa = data["per_arm"]
39      rl = pa["left"][KEY][0] / RAND["left"]
40      rr = pa["right"][KEY][0] / RAND["right"]

```

```

41     avg = (rl + rr) / 2.0
42     data.setdefault("average_relative_to_random", {})
43     data["average_relative_to_random"][KEY] = {
44         "left_ratio": rl, "right_ratio": rr, "avg_ratio": avg,
45         "rel_pct": (avg - 1.0) * 100.0,
46         "baseline_random": {"left": RAND["left"], "right": RAND["right"]},
47     }
48     json.dump(data, open(p, "w", encoding="utf-8"), indent=4, ensure_ascii=False)
49     return rl, rr, avg
50
51
52 def main():
53     print(f"{'方法':<16s}{'左/rand':>9s}{'右/rand':>9s}{'比值平均':>9s}{'相对随机':>11s}")
54     print("-" * 56)
55     for label, d in EXPS:
56         if d is None:
57             rl = rr = avg = 1.0
58         else:
59             if not os.path.exists(os.path.join(OUT, d, "experiment_results.json")):
60                 print(f"{label:<16s} (缺 {d})")
61                 continue
62             rl, rr, avg = process(d)
63             tag = "" if avg >= 1.0 else " (优于随机)"
64             print(f"{label:<16s}{rl:9.3f}{rr:9.3f}{avg:9.3f}{(avg-1)*100:+10.1f}%{tag}")
65             print("\n[写回] 各 JSON 已新增 average_relative_to_random 字段(原 average 保留不动)")
66
67
68 if __name__ == "__main__":
69     main()

```

visualize_coreseset.py ——核心集散点、CRAIG 代表簇分布与动作分布等可视化

```

1  """
2  核心集可视化（用于报告图文并茂）：
3      1) 把全部 ~20000 帧用 CLIP 图像特征做 2D 投影(PCA)，每种方法单独一张散点图，
4          灰色=全部帧，彩色=该方法最终选中的核心集帧 -> 看选中帧的分布。
5          方法：random / 度量一(facility 仅动作) / 度量二(facility 动作+视觉) / 度量三(CRAIG 梯度)。
6      2) 把效果最好的 CRAIG 核心集里抽若干帧解码出来拼成一张图(看选中的是什么画面)。
7      3) 左右臂动作速度分布直方图(证据：右臂重尾)。
8      4) CRAIG 代表簇分布：在梯度空间把全池每帧划给最近的选中点(代表点)，代表点按其
9          "名下帧数"(代表簇质量 w_j)着色与缩放 -> 直观呈现"代表簇重加权"这一步，并配
10         簇质量直方图(长尾说明少数代表点折叠了大量冗余帧)。
11
12  输出统一存到 figures_vca/(报告直接引用)。默认用右臂(CRAIG 在右臂取胜，最有代表性)。
13  运行：python visualize_coreseset.py [arm] # arm = right(默认) | left
14  注意：会重建惊奇度矩阵 + 训练 pilot，耗时(约十几分钟)。
15  """
16  import os

```

```

17 import sys
18 import json
19 import argparse
20 import numpy as np
21 import matplotlib
22 matplotlib.use("Agg")
23 import matplotlib.pyplot as plt
24
25 import config as cfg
26 from pairwise import build_pairwise_M, build_grad_M, global_quantile_from_M
27 from coreset import pcras_select, random_select
28 from mlp import pilot_gradient_features
29
30 OUT = r"E:\vision-roc\模式识别课设LaTeX模板\figures_vca"
31 VID = r"E:\vision-roc\data_raw\videos\observation.images.top\chunk-000\file-000.mp4"
32 COLORS = {"random": "#6fa8dc", "m1": "#e6b8af", "m2": "#e69138", "m3": "#5a876d"}
33 NAMES = {"random": "Random (10%)", "m1": "(1) action dist.",
34          "m2": "(2) action+vision dist.", "m3": "(3) gradient dist. (CRAIG)"}
35
36
37 def pca_2d(X, eps=1e-8):
38     """numpy 实现的 2D PCA(无 sklearn 依赖)。"""
39     Xc = X.astype(np.float64) - X.astype(np.float64).mean(0)
40     # 用协方差特征分解, 取前两主成分
41     cov = Xc.T @ Xc
42     w, V = np.linalg.eigh(cov)
43     comps = V[:, ::-1][:, :2] # 最大两个特征向量
44     return Xc @ comps
45
46
47 def select_indices(arm, img, txt, action, ep_id, frame_id, tr):
48     """返回 {方法: 选中的全局帧索引数组}。"""
49     N = len(img)
50     action7 = action[:, 7:] if arm == "right" else action[:, :7]
51     F_mlp = np.concatenate([img, np.tile(txt, (N, 1))], axis=1)
52     img_tr, act_tr = img[tr], action7[tr]
53     ep_tr, fr_tr = ep_id[tr], frame_id[tr]
54     B = round(cfg.BUDGET_RATIO * len(tr))
55     mpath = cfg.M_PATH.replace(".dat", f"_vis_{arm}.dat")
56     out = {}
57
58     # random (取 seed 0)
59     out["random"] = tr[random_select(len(tr), B, seed=0)]
60
61     # 度量一: facility, 仅动作(mode2)
62     M = build_pairwise_M(img_tr, act_tr, mpath.replace(".dat", "_a.dat"), 2, cfg.W_ACT, cfg.W_VIS,
63                          cfg.M_BLOCK, cfg.EPS,
64                          ep_id=ep_tr, frame_id=fr_tr, standardize_act=cfg.STANDARDIZE_ACT,

```

```

64         use_dynamics=cfg.USE_DYNAMICS, w_pos=cfg.W_POS, w_vel=cfg.W_VEL, w_acc=cfg
        .W_ACC)
65     thr = global_quantile_from_M(M, cfg.Q_PERCENTILE); sig = global_quantile_from_M(M, cfg.SIGMA_Q)
66     sel, _ = pcras_select(M, act_tr, ep_tr, fr_tr, cfg.BUDGET_RATIO, cfg.GRASP_THR, thr, cfg.EPS,
67         method="facility", sigma=sig)
68     out["m1"] = tr[sel]
69
70     # 度量二: facility, 动作+视觉(model)
71     M = build_pairwise_M(img_tr, act_tr, mpath.replace(".dat", "_av.dat"), 1, cfg.W_ACT, cfg.W_VIS,
72         cfg.M_BLOCK, cfg.EPS,
73         ep_id=ep_tr, frame_id=fr_tr, standardize_act=cfg.STANDARDIZE_ACT,
74         use_dynamics=cfg.USE_DYNAMICS, w_pos=cfg.W_POS, w_vel=cfg.W_VEL, w_acc=cfg
75         .W_ACC)
76     thr = global_quantile_from_M(M, cfg.Q_PERCENTILE); sig = global_quantile_from_M(M, cfg.SIGMA_Q)
77     sel, _ = pcras_select(M, act_tr, ep_tr, fr_tr, cfg.BUDGET_RATIO, cfg.GRASP_THR, thr, cfg.EPS,
78         method="facility", sigma=sig)
79     out["m2"] = tr[sel]
80
81     # 度量三: CRAIG, 梯度空间
82     G = pilot_gradient_features(F_mlp[tr], action7[tr], cfg, seed=0)
83     Mg = build_grad_M(G, mpath.replace(".dat", "_grad.dat"), cfg.M_BLOCK, cfg.EPS)
84     thr = global_quantile_from_M(Mg, cfg.Q_PERCENTILE); sig = global_quantile_from_M(Mg, cfg.
85         SIGMA_Q)
86     sel, _ = pcras_select(Mg, act_tr, ep_tr, fr_tr, cfg.BUDGET_RATIO, cfg.GRASP_THR, thr, cfg.EPS,
87         method="facility", sigma=sig)
88     out["m3"] = tr[sel]
89
90     # 额外回传 CRAIG 内部量, 供"代表簇分布"可视化复用(免去再训一次 pilot)
91     craig_ctx = dict(G=G, Mg=Mg, sel_local=np.asarray(sel, dtype=np.int64), sigma=sig)
92     return out, craig_ctx
93
94 def plot_scatters(emb, sel, arm):
95     """每种方法单独一张: 灰色全部点 + 彩色选中点."""
96     for key in ["random", "m1", "m2", "m3"]:
97         idx = sel[key]
98         fig, ax = plt.subplots(figsize=(5.2, 4.6))
99         ax.scatter(emb[:, 0], emb[:, 1], s=4, c="#cfcfcf", alpha=0.5, linewidths=0, label="all
100             frames")
101         ax.scatter(emb[idx, 0], emb[idx, 1], s=11, c=COLORS[key], alpha=0.9,
102             edgecolors="k", linewidths=0.2, label=f"selected coresets ({len(idx)})")
103         ax.set_title(f"{NAMES[key]} — {arm} arm", fontsize=11)
104         ax.set_xticks([]); ax.set_yticks([])
105         ax.legend(fontsize=9, loc="upper right", framealpha=0.9)
106         plt.tight_layout()
107         p = os.path.join(OUT, f"scatter_{key}_{arm}.png")
108         plt.savefig(p, dpi=150); plt.close()
109         print("saved", p)

```

```

107
108 def craig_assignment(M, selected, sigma, eps=1e-8):
109     """复刻 coreset.craig_weights 的归簇逻辑，额外回传每帧归属。
110     把全池每帧划给"梯度相似度最大"的选中点(代表点)；返回：
111         assign —— 每帧归属的代表点在 selected 中的位置(长度 N)；
112         w —— 各代表点权重 = 其名下帧数，归一到均值 1(即代表簇质量，长度 = |selected|)。"""
113     sel = np.asarray(selected, dtype=np.int64)
114     N = M.shape[0]
115     if sigma is None or sigma <= 0:
116         rng = np.random.default_rng(0)
117         s = M[rng.choice(N, size=min(N, 1000), replace=False)].astype(np.float64)
118         s = s[s > 0]
119         sigma = float(np.median(s)) if s.size else 1.0
120     best_sim = np.full(N, -np.inf, dtype=np.float64)
121     assign = np.zeros(N, dtype=np.int64)
122     for pos, j in enumerate(sel):
123         sim_j = np.exp(-(M[int(j)].astype(np.float64) / sigma) ** 2)
124         better = sim_j > best_sim
125         best_sim[better] = sim_j[better]
126         assign[better] = pos
127     counts = np.bincount(assign, minlength=len(sel)).astype(np.float64)
128     counts = np.maximum(counts, eps)
129     w = counts * (len(sel) / counts.sum()) # 归一到均值 1
130     return assign, w
131
132
133 def plot_craig_clusters(ctx, arm):
134     """CRAIG 代表簇分布可视化(对应方法图片一的"代表簇重加权"一步):
135     (a) 梯度空间 2D PCA 上，灰点为全池帧，选中的代表点按代表簇质量 w_j 着色+缩放；
136     (b) 代表簇质量 w_j 的直方图(长尾 = 少数代表点折叠了大量冗余帧)。"""
137     G, Mg, sel = ctx["G"], ctx["Mg"], ctx["sel_local"]
138     assign, w = craig_assignment(Mg, sel, ctx["sigma"])
139     emb = pca_2d(G) # 在梯度空间投影，簇结构最清晰
140
141     fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(10.6, 4.4),
142                                   gridspec_kw={"width_ratios": [1.4, 1.0]})
143     # (a) 全池帧(灰) + 代表点(按簇质量着色/缩放)
144     ax0.scatter(emb[:, 0], emb[:, 1], s=4, c="#d2d2d2", alpha=0.5, linewidths=0, label="all pool
145               frames")
146     sc = ax0.scatter(emb[sel, 0], emb[sel, 1], s=8 + 26 * (w / w.max()), c=w,
147                    cmap="viridis", alpha=0.95, edgecolors="k", linewidths=0.25)
148     cb = fig.colorbar(sc, ax=ax0, fraction=0.046, pad=0.02)
149     cb.set_label(r"cluster mass $w_j$ (mean$=1$)", fontsize=9)
150     ax0.set_title(f"(a) CRAIG representative clusters — {arm} arm", fontsize=11)
151     ax0.set_xticks([]); ax0.set_yticks([])
152     ax0.legend(fontsize=9, loc="upper right", framealpha=0.9)
153     # (b) 簇质量直方图(长尾)
154     ax1.hist(w, bins=30, color="#5a876d", alpha=0.85, edgecolor="white", linewidth=0.4)

```

```

154     ax1.axvline(1.0, color="k", ls="--", lw=1.0, label="mean = 1")
155     ax1.set_yscale("log")
156     ax1.set_xlabel(r"cluster mass $w_j$"); ax1.set_ylabel("# representatives (log)")
157     ax1.set_title("(b) distribution of cluster mass", fontsize=11)
158     ax1.legend(fontsize=9)
159     plt.tight_layout()
160     p = os.path.join(OUT, f"craig_clusters_{arm}.png")
161     plt.savefig(p, dpi=150); plt.close()
162     print("saved", p)
163
164
165 def plot_action_dist(action, ep_id, frame_id):
166     """左右臂动作速度(逐 episode  $\Delta$ 动作范数)分布直方图: 右臂更重尾。"""
167     def speeds(a7):
168         s = []
169         for e in np.unique(ep_id):
170             o = np.where(ep_id == e)[0]; o = o[np.argsort(frame_id[o])]
171             v = np.linalg.norm(np.diff(a7[o], axis=0), axis=1)
172             s.append(v)
173         return np.concatenate(s)
174     sl = speeds(action[:, :7]); sr = speeds(action[:, 7:])
175     fig, ax = plt.subplots(figsize=(5.6, 3.8))
176     bins = np.linspace(0, max(sl.max(), sr.max()), 60)
177     ax.hist(sl, bins=bins, alpha=0.6, label="left arm", color="#6fa8dc")
178     ax.hist(sr, bins=bins, alpha=0.6, label="right arm", color="#5a876d")
179     ax.set_yscale("log"); ax.set_xlabel("inter-frame action speed |delta action|"); ax.set_ylabel(
180         ("# frames (log)")
181     )
182     ax.legend(fontsize=10); plt.tight_layout()
183     p = os.path.join(OUT, "action_dist_arms.png"); plt.savefig(p, dpi=150); plt.close()
184     print("saved", p)
185
186
187 def montage_craig_frames(craig_global_idx, n=24, cols=6):
188     """从 CRAIG 核心集里均匀抽 n 帧解码拼图。"""
189     sys.path.insert(0, r"E:\vision-roc\pylibs")
190     import av
191     from PIL import Image, ImageDraw, ImageFont
192     pick = sorted(np.array(craig_global_idx)[np.linspace(0, len(craig_global_idx) - 1, n).astype(
193         int)].tolist())
194     try:
195         font = ImageFont.truetype("arial.ttf", 22)
196     except Exception:
197         font = ImageFont.load_default()
198     cont = av.open(VID); st = cont.streams.video[0]
199     cur = 0; ti = 0; imgs = {}
200     for fr in cont.decode(st):
201         if ti >= len(pick):
202             break

```

```

200         if cur == pick[ti]:
201             im = Image.fromarray(fr.to_ndarray(format="rgb24")).resize((360, 270))
202             d = ImageDraw.Draw(im)
203             d.rectangle([4, 4, 150, 32], fill=(0, 0, 0))
204             d.text((8, 6), f"frame {cur}", fill=(255, 255, 0), font=font)
205             imgs[cur] = im; ti += 1
206         cur += 1
207     cont.close()
208     keys = sorted(imgs.keys()); rows = int(np.ceil(len(keys) / cols))
209     canvas = Image.new("RGB", (cols * 360, rows * 270), (30, 30, 30))
210     for i, k in enumerate(keys):
211         canvas.paste(imgs[k], ((i % cols) * 360, (i // cols) * 270))
212     p = os.path.join(OUT, "craig_coreset_frames.png"); canvas.save(p)
213     print("saved", p)
214
215
216 def main():
217     ap = argparse.ArgumentParser()
218     ap.add_argument("arm", nargs="?", default="right", choices=["left", "right"])
219     arm = ap.parse_args().arm
220     os.makedirs(OUT, exist_ok=True)
221
222     d = np.load(cfg.FEATURE_PATH)
223     img = d["img_feat"].astype(np.float32); txt = d["txt_feat"].astype(np.float32)
224     action = d["action"].astype(np.float32)
225     ep_id = d["ep_id"].astype(int); frame_id = d["frame_id"].astype(int)
226
227     rng = np.random.default_rng(cfg.SEED)
228     test_eps = set(rng.choice(np.unique(ep_id), size=cfg.N_TEST_EP, replace=False).tolist())
229     tr = np.where(~np.isin(ep_id, list(test_eps)))[0]
230
231     print("[1/4] 2D PCA embedding of all frames ...")
232     emb = pca_2d(img)
233
234     print("[2/5] computing selected coresets per method ...")
235     sel, craig_ctx = select_indices(arm, img, txt, action, ep_id, frame_id, tr)
236     json.dump({k: v.tolist() for k, v in sel.items()},
237              open(os.path.join(OUT, f"selected_idx_{arm}.json"), "w"))
238
239     print("[3/5] scatter plots + action-distribution ...")
240     plot_scatters(emb, sel, arm)
241     plot_action_dist(action, ep_id, frame_id)
242
243     print("[4/5] CRAIG representative-cluster distribution ...")
244     plot_craig_clusters(craig_ctx, arm)
245
246     print("[5/5] CRAIG coreset frame montage ...")
247     montage_craig_frames(sel["m3"])

```

```

248
249     # 清理临时惊奇度矩阵(各 ~648MB)
250     import gc; gc.collect()
251     for suf in ("_a", "_av", "_grad"):
252         pth = cfg.M_PATH.replace(".dat", f"_vis_{arm}{suf}.dat")
253         try:
254             os.remove(pth)
255         except OSError:
256             pass
257     print("ALL DONE ->", OUT)
258
259
260 if __name__ == "__main__":
261     main()

```

plot_results_bar.py ——主结果柱状图（相对随机比值口径）

```

1  """
2  重画主结果柱状图：横轴各方法，每方法三柱(左臂/右臂/两臂平均)，纵轴为【相对随机的 MSE 比值】
3  (每臂先除以该臂随机基线；随机自身=1.0)。在 y=1.0 处画随机基线虚线，<1 即优于随机。
4  口径与报告主表一致(比值平均)。输出：figures_vca/results_bar.pdf
5  运行：python plot_results_bar.py
6  """
7
8  import sys
9
10 sys.path.insert(0, r"E:\vision-roc\pylibs")
11
12 import os
13
14 os.environ.setdefault("MPLCONFIGDIR", r"E:\vision-roc\.cache\matplotlib")
15
16 import numpy as np
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21
22 import matplotlib.pyplot as plt
23
24
25 plt.rcParams["font.sans-serif"] = ["SimHei"]
26 plt.rcParams["axes.unicode_minus"] = False
27
28
29 # 各方法左右臂 MSE(x1e-3，取自报告主表)与随机基线
30 RAND = {"left": 3.901, "right": 12.429}
31
32 METHODS = [
33     ("全量(100%)", 3.761, 11.167),
34     ("随机(10%)", 3.901, 12.429),
35     ("动作+视觉\n(惊奇度)", 3.387, 13.657),
36     ("梯度\nCRAIG", 3.972, 11.569),
37 ]
38
39 labels, rl, rr, ra = [], [], [], []
40
41 for name, l, r in METHODS:
42     labels.append(name)
43     a = 1 / RAND["left"]

```

```

32     b = r / RAND["right"]
33     rl.append(a); rr.append(b); ra.append((a + b) / 2)
34
35 x = np.arange(len(labels))
36 w = 0.26
37 fig, ax = plt.subplots(figsize=(8.2, 4.2))
38 b1 = ax.bar(x - w, rl, w, label="左臂", color="#6baed6")
39 b2 = ax.bar(x, rr, w, label="右臂", color="#fd8d3c")
40 b3 = ax.bar(x + w, ra, w, label="两臂平均", color="#74c476")
41
42 ax.axhline(1.0, ls="--", lw=1.2, color="#888", zorder=0)
43 ax.text(len(labels) - 0.5, 1.005, "随机基线 =1.0", color="#666", fontsize=9, ha="right", va="bottom
    ")
44
45 for bars in (b1, b2, b3):
46     for rect in bars:
47         h = rect.get_height()
48         ax.text(rect.get_x() + rect.get_width() / 2, h + 0.006, f"{h:.3f}",
49                 ha="center", va="bottom", fontsize=7)
50
51 ax.set_ylabel("相对随机的 MSE 比值 (<1 更优)")
52 ax.set_xticks(x)
53 ax.set_xticklabels(labels, fontsize=10)
54 ax.set_ylim(0.80, max(rr) + 0.10)
55 ax.legend(loc="upper left", ncol=3, frameon=False)
56 ax.set_title("各方法相对随机基线的动作预测 MSE 比值 (越低越好)")
57 fig.tight_layout()
58
59 out = r"E:\vision-roc\模式识别课设LaTeX模板\figures_vca\results_bar.pdf"
60 fig.savefig(out, bbox_inches="tight")
61 print(f"[saved] {out}")
62 for name, a, b, c in zip(labels, rl, rr, ra):
63     print(f" {name.replace(chr(10), ' '):16s} 左{a:.3f} 右{b:.3f} 平均{c:.3f}")

```

参考文献

- [1] Brohan A, Brown N, Carbajal J, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818, 2023.
- [2] Kim M J, Pertsch K, Karamcheti S, et al. Openvla: An open-source vision-language-action model. Conference on Robot Learning (CoRL). 2024.
- [3] Zhao T Z, Kumar V, Levine S, et al. Learning fine-grained bimanual manipulation with low-cost hardware. Robotics: Science and Systems (RSS), 2023.
- [4] Cadene R, Alibert S, Soare A, et al. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch[OL]. 2024. <https://github.com/huggingface/lerobot>.
- [5] Rao R P, Ballard D H. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. Nature Neuroscience, 1999, 2(1): 79-87.
- [6] Moruzzi G, Magoun H W. Brain stem reticular formation and activation of the eeg. Electroencephalography and Clinical Neurophysiology, 1949, 1(1-4): 455-473.
- [7] Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and rt-x models. arXiv preprint arXiv:2310.08864, 2023.
- [8] Black K, Brown N, Driess D, et al. π_0 : A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164, 2024.
- [9] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing[OL]. 2026. <https://github.com/starVLA/starVLA>.
- [10] Wang S, Shi J, Fu Z, et al. World action models: The next frontier in embodied ai. arXiv preprint arXiv:2605.12090, 2026.
- [11] Qi C R, Yi L, Su H, et al. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. Advances in Neural Information Processing Systems (NeurIPS). 2017.

-
- [12] Sener O, Savarese S. Active learning for convolutional neural networks: A core-set approach. International Conference on Learning Representations (ICLR). 2018.
- [13] Kaufman L, Rousseeuw P J. Finding groups in data: an introduction to cluster analysis[]. John Wiley & Sons, 1990.
- [14] Nemhauser G L, Wolsey L A, Fisher M L. An analysis of approximations for maximizing submodular set functions. Mathematical Programming, 1978, 14(1): 265-294.
- [15] Leskovec J, Krause A, Guestrin C, et al. Cost-effective outbreak detection in networks. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2007: 420-429.
- [16] Toneva M, Sordoni A, Combes R T d, et al. An empirical study of example forgetting during deep neural network learning. International Conference on Learning Representations (ICLR). 2019.
- [17] Paul M, Ganguli S, Dziugaite G K. Deep learning on a data diet: Finding important examples early in training. Advances in Neural Information Processing Systems (NeurIPS). 2021.
- [18] Zheng H, Liu R, Lai F, et al. Coverage-centric coreset selection for high pruning rates. International Conference on Learning Representations (ICLR). 2023.
- [19] Mirzasoleiman B, Bilmes J, Leskovec J. Coresets for data-efficient training of machine learning models. International Conference on Machine Learning (ICML). 2020: 6950-6960.
- [20] Killamsetty K, Sivasubramanian D, Ramakrishnan G, et al. Grad-match: Gradient matching based data subset selection for efficient deep model training. International Conference on Machine Learning (ICML). 2021: 5464-5474.
- [21] Dass S, Khaddaj A, Engstrom L, et al. Datamil: Selecting data for robot imitation learning with datamodels. arXiv preprint arXiv:2505.09603, 2025.

-
- [22] Friston K. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 2010, 11(2): 127-138.
- [23] Itti L, Baldi P. Bayesian surprise attracts human attention. *Vision Research*, 2009, 49(10): 1295-1306.
- [24] Schultz W, Dayan P, Montague P R. A neural substrate of prediction and reward. *Science*, 1997, 275(5306): 1593-1599.
- [25] Desimone R, Duncan J. Neural mechanisms of selective visual attention. *Annual Review of Neuroscience*, 1995, 18: 193-222.
- [26] McClelland J L, McNaughton B L, O'Reilly R C. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 1995, 102(3): 419-457.
- [27] Radford A, Kim J W, Hallacy C, et al. Learning transferable visual models from natural language supervision. *International Conference on Machine Learning (ICML)*. 2021: 8748-8763.
- [28] Meta AI Research. Sam 3: Segment anything with concepts. *arXiv preprint*, 2025.